CLASSROOM TRAINING HANDBOOK ED. 2026 · V1.0

Ansible Automation

Complete Trainer Notes & Practical Lab Manual

A comprehensive classroom teaching handbook for beginner-level Linux students transitioning into infrastructure automation. Covers Ansible 2.16 (Core) from first principles to production-ready roles, with bilingual English-Malayalam explanations, hands-on labs, capstone project, interview question bank, and viva preparation.

LAB ENVIRONMENT

ansible-control	Ubuntu 24.04 · 192.168.1.10
webserver1	AlmaLinux 9 · 192.168.1.21
webserver2	AlmaLinux 9 · 192.168.1.22

PREPARED BY

Senior Linux System Administrator & Trainer

ANSIBLE-CORE 2.16

BEGINNER — INTERMEDIATE

Table of Contents

Introduction to Automation & Ansible 11

1.1 What is IT Automation? . . . 11
1.2 Why Ansible? Key Features . . . 12
1.3 Agentless vs Agent-Based Tools . . . 12
1.4 The Push Model vs Pull Model . . . 13
1.5 Idempotency — The Most Important Concept . . . 13
1.6 YAML — The Language of Ansible . . . 14
1.7 Where Ansible Fits in DevOps . . . 14

Lab Environment Setup 16

2.1 Lab Topology Overview . . . 16
2.2 Setting Hostnames . . . 17
2.3 Configuring /etc/hosts for Name Resolution . . . 17
2.4 Creating the ansible User on Managed Nodes . . . 18
2.5 Verifying SSH Connectivity . . . 19
2.6 Lab Topology Diagram . . . 19

Installing and Configuring Ansible 2.16 21

3.1 Installation Methods Overview . . . 21
3.2 Installing ansible-core 2.16 via pip . . . 21
3.3 The ansible.cfg Configuration File . . . 22
3.4 Configuration Precedence (Four Levels) . . . 23
3.5 Creating a Project-Level ansible.cfg . . . 23
3.6 Verifying the Configuration . . . 24

SSH, Sudo Privilege Escalation, and Security 26

4.1 Why SSH Key Authentication? . . . 26
4.2 Generating an SSH Key Pair . . . 26
4.3 Installing the Public Key on Managed Nodes . . . 27
4.4 Configuring SSH to Use the Key Automatically . . . 28
4.5 Sudoers Configuration (Recap and Production Hardening) . . . 28

4.6 Ansible become Directives . . . 29
4.7 Secure Credential Handling Best Practices . . . 30

Inventory Management **31**

5.1 What is an Inventory? . . . 31
5.2 The INI Inventory Format (Most Common) . . . 31
5.3 Common Inventory Parameters . . . 32
5.4 The YAML Inventory Format . . . 32
5.5 Host Variables vs Group Variables . . . 33
5.6 Verifying the Inventory . . . 34
5.7 The Default Inventory Locations . . . 34

Ad-Hoc Commands **36**

6.1 What is an Ad-Hoc Command? . . . 36
6.2 Anatomy of an Ad-Hoc Command . . . 36
6.3 Common Ad-Hoc Patterns . . . 37
Pattern 1: Run a command on remote hosts . . . 37
Pattern 2: Install a package (idempotent) . . . 37
Pattern 3: Start and enable a service . . . 37
Pattern 4: Create a user . . . 37
Pattern 5: Copy a file to remote hosts . . . 37
Pattern 6: Gather facts (system information) . . . 37
6.4 Useful Ad-Hoc Flags . . . 38
6.5 Verbosity Levels Explained . . . 38
6.6 Common Mistakes with Ad-Hoc Commands . . . 38

Ansible Modules — Deep Dive **40**

7.1 What is a Module? . . . 40
7.2 System Modules . . . 40
user — Create and manage users . . . 40
group — Create and manage groups . . . 41
service / systemd — Manage services . . . 41
hostname — Set the system hostname . . . 41
reboot — Reboot and wait for the host to come back . . . 41

7.3 Package Modules . . . 41
yum / dnf — RPM package management (AlmaLinux, RHEL, Rocky) . . . 41
apt — Debian package management (Ubuntu, Debian) . . . 42
pip — Python package management . . . 42
7.4 File Modules . . . 42
file — Set file properties (ownership, permissions, state) . . . 42
copy — Copy a file from control node to managed hosts . . . 42
lineinfile — Manage a single line in a file (idempotent) . . . 42
blockinfile — Manage a block of lines . . . 43
fetch — Copy a file from managed host to control node . . . 43
synchronize — rsync wrapper for efficient file sync . . . 43
7.5 Command Modules — Choosing the Right One . . . 43
7.6 Network and Firewall Modules . . . 44
firewalld — Manage firewalld zones and rules (RHEL family) . . . 44
ufw — Manage UFW rules (Debian family) . . . 44
7.7 Cloud and Source Control Modules . . . 44
git — Clone or update a git repository on remote hosts . . . 44
7.8 Quick Module Selection Guide . . . 44

Playbooks Foundations 47

8.1 What is a Playbook? . . . 47
8.2 Anatomy of a Playbook . . . 47
8.3 Running a Playbook . . . 48
8.4 Idempotency in Action — Run It Twice . . . 49
8.5 Check Mode (Dry Run) . . . 49
8.6 Diff Mode . . . 49
8.7 Syntax Validation . . . 49
8.8 Common Playbook Mistakes . . . 50

Variables, Facts, and Magic Variables 51

9.1 Why Variables? . . . 51
9.2 Defining Variables — Where and How . . . 51
9.3 Variable Precedence Table (Simplified) . . . 51
9.4 Defining Variables in a Playbook . . . 52

9.5 Loading Variables from Files (vars_files) . . . 52
9.6 Interactive Prompts (vars_prompt) . . . 53
9.7 Register — Capturing Task Output . . . 53
9.8 Magic Variables . . . 54
9.9 Facts — System Information from setup Module . . . 55
9.10 Using Facts in a Playbook . . . 55
9.11 Disabling Fact Gathering for Speed . . . 56
9.12 Custom Facts (facts.d) . . . 56

Conditionals, Loops, and Task Control **58**

10.1 The when Conditional . . . 58
10.2 Combining Conditions . . . 59
10.3 when with Registered Variables . . . 59
10.4 Loops — The loop Keyword . . . 59
10.5 Looping Over Dictionaries . . . 60
10.6 Looping Over Files (with_fileglob) . . . 60
10.7 until — Retry Until a Condition is Met . . . 60
10.8 Tags — Running Subsets of a Playbook . . . 61
10.9 block/rescue/always — Error Handling . . . 61
10.10 changed_when, failed_when, ignore_errors . . . 62

Templates with Jinja2 **64**

11.1 What is Jinja2? . . . 64
11.2 Jinja2 Syntax Basics . . . 64
11.3 Common Filters . . . 64
11.4 Creating a Template File . . . 65
11.5 Rendering the Template with the template Module . . . 66
11.6 Conditionals Inside Templates . . . 66
11.7 Loops Inside Templates . . . 67
11.8 Generating /etc/motd with System Info . . . 67
11.9 Validating Template Syntax . . . 68

Handlers and Error Handling **69**

12.1 What is a Handler? . . . 69

12.2 Defining and Notifying Handlers . . . 69
12.3 Handler Ordering — A Common Pitfall . . . 70
12.4 Forcing Handler Execution Early . . . 70
12.5 Forcing Handlers on Failure . . . 70
12.6 Error Handling with block/rescue/always . . . 71
12.7 Controlling Failure Behaviour . . . 71
12.8 Debugging with the debug Module . . . 72

Ansible Roles and Galaxy 74

13.1 Why Roles? . . . 74
13.2 Role Directory Layout . . . 74
13.3 Creating a Role with ansible-galaxy init . . . 75
13.4 Building a Reusable nginx Role . . . 75
defaults/main.yml — Sensible defaults . . . 76
tasks/main.yml — The main task list . . . 76
handlers/main.yml . . . 77
templates/index.html.j2 . . . 77
templates/nginx.conf.j2 . . . 77
13.5 Using a Role in a Playbook . . . 78
13.6 Role Dependencies . . . 78
13.7 Ansible Galaxy — The Role Marketplace . . . 79
Searching and installing a role . . . 79
requirements.yml — Managing Role Dependencies . . . 79
13.8 Roles vs Collections . . . 80

Ansible Vault — Securing Secrets 81

14.1 The Problem: Plaintext Secrets in Playbooks . . . 81
14.2 Creating an Encrypted File . . . 81
14.3 Viewing an Encrypted File . . . 82
14.4 Editing an Encrypted File . . . 82
14.5 Encrypting an Existing File . . . 82
14.6 Decrypting an Encrypted File . . . 82
14.7 Changing the Vault Password (rekey) . . . 82
14.8 encrypt_string — Inline Secret Encryption . . . 83

14.9 Vault Password Files — Non-Interactive Decryption . . . 83
14.10 Multiple Vault Passwords . . . 83
14.11 CI/CD Integration Best Practices . . . 84

Practical Lab Tasks (8 Labs) 85

Lab 1 — Inventory Setup & First Ad-Hoc Ping . . . 85
Prerequisites . . . 85
Procedure . . . 85
Expected Result . . . 85
Lab 2 — Install and Configure nginx on Webservers . . . 86
Prerequisites . . . 86
Procedure . . . 86
Validation . . . 86
Lab 3 — Create Users with SSH Keys via Playbook . . . 87
Objective . . . 87
Procedure . . . 87
Validation . . . 87
Lab 4 — Use Variables and Templates to Customise index.html per Host . . . 87
Objective . . . 87
Procedure . . . 87
Validation . . . 88
Lab 5 — Conditional Install: httpd on Alma, apache2 on Ubuntu . . . 88
Objective . . . 88
Procedure . . . 88
Validation . . . 89
Lab 6 — Build a Reusable nginx Role . . . 89
Objective . . . 89
Procedure . . . 89
Validation . . . 91
Lab 7 — Use Vault to Encrypt a Database Password . . . 91
Objective . . . 91
Procedure . . . 91
Validation . . . 91

Lab 8 — Schedule a Cron Job Across the Fleet with Handlers . . . 91
Objective . . . 91
Procedure . . . 91
Validation . . . 92

Capstone Project: LAMP Stack Deployment 94

16.1 Project Goals . . . 94
16.2 Architecture . . . 94
16.3 Project Directory Layout . . . 94
16.4 Variables Plan . . . 95
16.5 The Top-Level Playbook (site.yml) . . . 95
16.6 The apache Role . . . 95
16.7 The mysql Role . . . 97
16.8 The php Role . . . 97
16.9 The app Role . . . 98
16.10 Running the Capstone . . . 98
16.11 Validation Tests . . . 99
16.12 Cleanup . . . 99

Interview Question Bank (40+ Questions) 101

A. Basic Concepts (10 Questions) . . . 101
Q1. What is Ansible and what problem does it solve? . . . 101
Q2. Why is Ansible called agentless? . . . 101
Q3. What is idempotency and why is it important? . . . 101
Q4. Explain the difference between push and pull configuration models. . . . 101
Q5. What language are Ansible playbooks written in? . . . 101
Q6. What is the difference between ansible-core and the ansible community package? . . . 101
Q7. What is a playbook? What is a play? What is a task? . . . 102
Q8. What is the difference between Ansible and Terraform? . . . 102
Q9. Name three advantages of Ansible over shell scripts. . . . 102
Q10. What is the Ansible control node and what are managed nodes? . . . 102
B. Inventory & Ad-Hoc Commands (8 Questions) . . . 102
Q11. What is an inventory file? . . . 102
Q12. What is the difference between [webservers] and [webservers:vars] in an INI inventory? . 102

Q13. What are group_vars and host_vars directories? . . . 102
Q14. Write an ad-hoc command to install nginx on all webservers using sudo. . . . 102
Q15. What is the difference between command and shell modules? . . . 102
Q16. What does the -v, -vv, -vvv, -vvvv progression do? . . . 102
Q17. How do you target only one host from a group on the command line? . . . 103
Q18. What does --check do? . . . 103
C. Playbooks & YAML (8 Questions) . . . 103
Q19. What is the structure of a playbook? . . . 103
Q20. Why must YAML use spaces instead of tabs? . . . 103
Q21. What is the difference between vars and vars_files? . . . 103
Q22. How do you validate a playbook before running it? . . . 103
Q23. What is the difference between roles: and include_role? . . . 103
Q24. What is the become directive and when would you use it? . . . 103
Q25. What is the purpose of name: keys in tasks? . . . 103
Q26. What is the difference between --check and --syntax-check? . . . 104
D. Modules & Variables (7 Questions) . . . 104
Q27. What is the difference between ansible.builtin.yum and ansible.builtin.dnf? . . . 104
Q28. What is the setup module used for? . . . 104
Q29. What is the register keyword for? . . . 104
Q30. Name three magic variables in Ansible. . . . 104
Q31. What is variable precedence in Ansible? . . . 104
Q32. What does the | default() filter do? . . . 104
Q33. What is the difference between copy and template modules? . . . 104
E. Roles & Galaxy (5 Questions) . . . 105
Q34. What is the standard role directory structure? . . . 105
Q35. How do you create a new role? . . . 105
Q36. What is Ansible Galaxy? . . . 105
Q37. What is a requirements.yml file? . . . 105
Q38. How do you declare that one role depends on another? . . . 105
F. Vault & Security (5 Questions) . . . 105
Q39. What is Ansible Vault and why would you use it? . . . 105
Q40. How do you run a playbook non-interactively with vault-encrypted files? . . . 105
Q41. What is encrypt_string used for? . . . 105

Q42. What are multiple vault IDs useful for? . . . 105
Q43. Name three security best practices for Ansible automation. . . . 106
Q44. What is the difference between ansible.builtin.ping and a normal ICMP ping? . . . 106
Q45. How do you debug a failing playbook? . . . 106

Viva Voce Questions (30) 107

A. Foundational (10) . . . 107
V1. Define Ansible in one sentence. . . . 107
V2. What protocol does Ansible use to communicate with Linux managed nodes? . . . 107
V3. Name the three levels of Ansible hierarchy. . . . 107
V4. What does idempotent mean? . . . 107
V5. Which file format are playbooks written in? . . . 107
V6. Name the three sections of ansible.cfg we configured in Chapter 3. . . . 107
V7. What is the difference between a control node and a managed node? . . . 107
V8. What is the default SSH user configured in our ansible.cfg? . . . 107
V9. Name the four levels of ansible.cfg precedence. . . . 107
V10. Which command shows the active Ansible configuration? . . . 107
B. Inventory & Commands (10) . . . 108
V11. What is the difference between [group] and [group:vars] in an INI inventory? . . . 108
V12. How do you verify an inventory file? . . . 108
V13. Write the ad-hoc pattern to ping all webservers. . . . 108
V14. What does the -b flag do in an ad-hoc command? . . . 108
V15. What is the difference between command and shell modules? . . . 108
V16. Which flag performs a dry run? . . . 108
V17. What does --limit do? . . . 108
V18. What is the raw module used for? . . . 108
V19. How do you check documentation for a module offline? . . . 108
V20. Name the four verbosity flags and what each adds. . . . 108
C. Playbooks, Variables, Roles (10) . . . 108
V21. What is the difference between vars and vars_files? . . . 108
V22. What does register do? . . . 108
V23. Name three magic variables. . . . 108
V24. Which fact tells you the OS family? . . . 109

V25. What does the when keyword do? . . . 109
V26. What is a handler? . . . 109
V27. What is the difference between roles: and include_role? . . . 109
V28. What is the purpose of defaults/main.yml in a role? . . . 109
V29. What is Ansible Vault? . . . 109
V30. What is the purpose of block/rescue/always? . . . 109

Troubleshooting Guide **110**

19.1 Connection Errors . . . 110
19.2 Sudo / Privilege Errors . . . 110
19.3 YAML Syntax Errors . . . 111
19.4 Python Errors on Managed Nodes . . . 111
19.5 Module Errors . . . 111
19.6 Idempotency Issues . . . 112
19.7 Verbose Debugging Workflow . . . 112
19.8 Common Fact-Gathering Issues . . . 112

Glossary of Ansible Terminology **114**

Quick Reference Cheat Sheet **117**

B.1 Essential CLI Commands . . . 117
B.2 Top 20 Modules (FQCN) . . . 118
B.3 Top 10 ansible-playbook Flags . . . 119
B.4 Top 10 Jinja2 Filters . . . 119
B.5 Common Magic Variables & Facts . . . 119
B.6 Essential ansible.cfg Sections . . . 120
B.7 Useful Ad-Hoc One-Liners . . . 120
B.8 Minimal Playbook Template . . . 121

CHAPTER 1

Introduction to Automation & Ansible

LEARNING OBJECTIVES

- *Define IT automation and explain why it is essential in modern infrastructure management.*
- *Identify the key differences between agentless and agent-based automation tools.*
- *Compare Ansible with Chef, Puppet, and Salt across architecture and use-case dimensions.*
- *Explain the concept of idempotency and why it matters for safe automation.*
- *Read and write basic YAML files used for Ansible configuration.*
- *Describe where Ansible fits within a typical DevOps toolchain.*

1.1 What is IT Automation?

IT automation is the practice of using software tools to perform repetitive operational tasks — such as installing packages, creating users, configuring services, deploying applications, and applying security patches — without manual intervention. In a traditional data centre, a system administrator would log in to each server over SSH and run commands one by one. This works for two or three servers, but it quickly becomes impossible when you have twenty, two hundred, or two thousand machines to manage. Automation tools solve this problem by letting you describe the desired state of your infrastructure in a file, and then having a tool enforce that state across all your servers consistently.

Beyond saving time, automation also improves reliability and reproducibility. A well-written automation script behaves identically every time it runs, on every server, which eliminates the small human errors that accumulate when administrators repeat commands by hand. It also serves as living documentation: the automation files themselves describe exactly how the systems are configured, which is far more trustworthy than out-of-date wiki pages. In modern DevOps practice, infrastructure-as-code (IaC) — the idea that your infrastructure configuration is version-controlled text — is the foundation of scalable operations.

MALAYALAM NOTE · Malayalam Teaching Note

Automation ennu parayunnathu computer-ukku parayunna commands human-ine kondu cheythittu alla, software vannu cheyyippikkunnathu aanu. Oru server-il oru kaaryam cheyyan nerittondu polum time venam, 100 server-il cheyyan aake impossible. Aakayirunnu automation tools undaakunnathu. Ansible ennathu aanu ithinte aazhcha easysthe tool kaaranam agent onnum install cheyyanda avashyam illa — just SSH mathi.

1.2 Why Ansible? Key Features

Ansible is an open-source automation tool created by Michael DeHaan in 2012 and acquired by Red Hat in 2015. It is now the most widely adopted automation tool in the Linux world and is part of the Red Hat Ansible Automation Platform product family. Ansible is popular because of five distinguishing features that make it beginner-friendly yet production-grade.

- **Agentless architecture.** Ansible does not require any special software (agent) to be installed on the managed servers. It uses standard SSH (Linux) or WinRM (Windows) to communicate, which means you can start automating existing servers without any prior setup.
- **Idempotent operations.** Ansible modules are designed so that running the same task multiple times produces the same end state. If a package is already installed, Ansible will not reinstall it; if a file already has the correct content, Ansible will not rewrite it.
- **Human-readable YAML.** Ansible playbooks are written in YAML, a simple data serialization language that is easy to read and write. No programming background is required to get started.
- **Push-based model.** The control node pushes configuration to the managed nodes. There is no need to set up a central server or agent polling mechanism, which keeps the architecture simple and predictable.
- **Module-rich ecosystem.** Ansible ships with hundreds of built-in modules (over 3,000 in the wider Ansible Galaxy community) covering everything from package management to cloud provisioning to network device configuration.

1.3 Agentless vs Agent-Based Tools

Automation tools broadly fall into two architectural categories. Agent-based tools like Chef, Puppet, and SaltStack require a small daemon (the agent) to be installed and continuously running on every managed server. The agent periodically contacts a master server, fetches the desired configuration, and applies it locally. Agentless tools like Ansible use existing management protocols (SSH, WinRM) to push configuration from a central control node, with no resident software on the target hosts.

Aspect	Agent-Based (Puppet, Chef, Salt)	Agentless (Ansible)
Setup effort on targets	Install and configure agent on every host	No agent — uses existing SSH/WinRM
Master server required	Yes, central master is mandatory	No master — control node runs Ansible directly
Configuration pull direction	Pull — agents poll master	Push — control node pushes to targets
Firewall requirements	Inbound port from agent to master	Outbound SSH (22) from control to target
Language used	DSL (Puppet), Ruby (Chef), YAML (Salt)	YAML (declarative, human-readable)
Speed at scale	Fast once agents are running	Slightly slower per-run, but simpler operationally
Best fit	Large, stable estates with central authority	Mixed estates, ad-hoc tasks, cloud environments

Table 1.1 — Agentless vs Agent-Based Automation Tools

MALAYALAM NOTE · Malayalam Teaching Note

Agentless ennu parayunnathu managed server-il onnum install cheyyanda avashyam illa ennu artham. SSH already server-il undaakum (Linux aanel), Ansible adhu vare communicate cheyyum. Agent-based tools-il (Puppet, Chef) oru small software install cheyyanam — adhine agent ennu parayum. Ithu setup time kooduthram, aanaal large-scale-il fast aayittu cheyyum. Ansible easy-to-start aanu, ithu aanu main advantage.

1.4 The Push Model vs Pull Model

The push model means the control node initiates the connection to managed nodes and pushes the desired state. The pull model means managed nodes periodically contact a central server to fetch the latest configuration. Each model has trade-offs. The push model is simpler to reason about (you trigger an action and immediately see results), but it requires the control node to have network access to every managed host. The pull model naturally scales to a large number of nodes that may be behind firewalls, but it introduces a lag between configuration change and enforcement, and it requires a highly-available master server.

Ansible uses the push model by default, which is one of the reasons it is so easy to learn. You run a command, Ansible SSHes into the targets, executes the tasks, and reports back. There is no daemon to start, no master to maintain, and no polling cycle to wait for. For environments that prefer the pull model (for example, very large estates where running Ansible from a single control node is impractical), Ansible offers *Ansible Pull*, a mode where each managed host periodically checks out a Git repository and runs a locally-targeted playbook. Pull mode is rarely used by beginners but is worth knowing about.

1.5 Idempotency — The Most Important Concept

Idempotency is a mathematical term that, in the context of automation, means: **running an operation multiple times has the same effect as running it once**. If a task is idempotent, you can safely re-run it after a failure, after a network blip, or simply to verify the system is still in the desired state — without fear of breaking anything or producing duplicate results.

For example, an idempotent task that says *ensure the user webadmin exists* will create the user the first time it runs, and on subsequent runs it will recognise that the user already exists and do nothing. A non-idempotent equivalent (useradd webadmin) would fail the second time with *user already exists*. Ansible modules are designed to be idempotent by default — you describe the desired state, and the module figures out whether any change is needed. This is what makes Ansible safe to re-run repeatedly in production.

TIP · Why idempotency matters in real operations

Imagine patching 500 servers. If your automation script is not idempotent and crashes halfway through, you cannot safely restart it — some servers may have received the patch twice, some once, some not at all, and your script may have created duplicate users or files. With an idempotent Ansible playbook, you simply re-run it. Ansible detects the existing state on each host and only changes what is needed. This is the single most powerful guarantee Ansible gives you.

MALAYALAM NOTE · Malayalam Teaching Note

Idempotency ennu parayunnathu — same task n-th time run cheyyiyalum result onnu maathramaayirikkum. Oru user create cheyyunna task n-th time run cheyyumbol, adhu already existing user-kku error kanikkilla — simply 'ok' ennu paranju ozhinnu pokum. Ithu production-il valiya kaaryam aanu — crash aanelum re-run cheyyam, duplicate users/files undaakilla.

1.6 YAML — The Language of Ansible

Ansible playbooks are written in YAML (YAML Ain't Markup Language), a human-friendly data serialization format. YAML uses indentation (spaces, not tabs) to define structure and key-value pairs to define data. Learning YAML takes about fifteen minutes, and you do not need to be a programmer to write playbooks. The most important YAML rules to remember are listed below.

- **Indentation is significant.** Use spaces (never tabs). Two spaces per level is the Ansible convention.
- **Key-value pairs use a colon.** Write key: value — the space after the colon is mandatory.
- **Lists use a leading dash.** Each list item starts with - at the same indentation level.
- **Dictionaries are nested key-value pairs.** They look like nested key-value lists.
- **Strings do not need quotes** unless they contain special characters like colons.
- **Comments start with #.**

A simple YAML example showing a list of fruits and a dictionary describing a person is shown below. Pay attention to the indentation — it defines the structure.

YAML

```yaml
# A list (sequence)
fruits:
  - apple
  - banana
  - orange

# A dictionary (mapping)
person:
  name: Anjali
  age: 28
  role: sysadmin
  skills:
    - linux
    - bash
    - ansible
```

1.7 Where Ansible Fits in DevOps

DevOps is a culture and set of practices that aim to shorten the development-to-operations cycle through automation, monitoring, and continuous feedback. Ansible occupies a critical position in this toolchain because it is the layer that translates infrastructure requirements into actual server state. Other tools in a typical DevOps stack include Git (version control), Jenkins or GitLab CI (continuous integration), Docker and Kubernetes (containerisation), and Prometheus or Grafana (monitoring). Ansible is used to provision the underlying servers, configure them, deploy the applications running on them, and orchestrate rolling updates or rollback.

DevOps Stage	Common Tools	Ansible Role
Source control	Git, GitHub, GitLab	Pulls playbooks and roles from Git
CI/CD pipeline	Jenkins, GitLab CI, GitHub Actions	Runs playbooks as a deployment step
Infrastructure provisioning	Terraform, Vagrant	Configures VMs after Terraform creates them
Configuration management	Ansible, Puppet, Chef, Salt	Ansible itself
Container orchestration	Kubernetes, OpenShift	Bootstraps nodes, deploys k8s itself
Monitoring & alerting	Prometheus, Grafana, Nagios	Installs and configures monitoring agents
Security & compliance	CIS-CAT, OpenSCAP	Enforces baseline security settings

Table 1.2 — Ansible within the DevOps Toolchain

CHAPTER SUMMARY

Automation replaces manual, repetitive server administration with declarative, version-controlled configuration. Ansible is an agentless, push-based, idempotent automation tool that uses YAML playbooks and standard SSH. Its simplicity and safety make it the most popular choice for Linux configuration management today. Idempotency — the guarantee that running a task multiple times has the same effect as running it once — is the single most important concept to internalise before writing playbooks.

REVIEW QUESTIONS

1. Define IT automation in your own words and give two examples of tasks that should be automated.
2. What does *agentless* mean in the context of Ansible? Why is this an advantage?
3. Compare the push and pull configuration models. Which one does Ansible use by default?
4. Explain idempotency with a concrete example that is NOT from this chapter.
5. List three differences between Ansible and Puppet.
6. Why is YAML chosen as the language for Ansible playbooks? Name two YAML rules.
7. Where does Ansible fit in a DevOps toolchain that also includes Git, Jenkins, and Kubernetes?

CHAPTER 2

Lab Environment Setup

LEARNING OBJECTIVES

- *Describe the three-node lab topology used throughout this manual.*
- *Configure hostnames and the /etc/hosts file on all three nodes.*
- *Verify network connectivity between control and managed nodes using ping and ssh.*
- *Create the dedicated ansible user account on managed nodes.*
- *Understand the security implications of using a non-root automation account.*

2.1 Lab Topology Overview

Throughout this manual we will use a consistent three-node lab environment. One machine acts as the Ansible control node — this is where Ansible is installed and where playbooks are executed from. The other two machines are managed nodes — these are the targets that Ansible will configure over SSH. In a real production environment you might have hundreds of managed nodes, but the workflow is identical; the only difference is the size of the inventory file.

Role	Hostname	OS	IP Address	Purpose
Control Node	ansible-control	Ubuntu 24.04 LTS	192.168.1.10	Runs Ansible, holds playbooks & inventory
Managed Node 1	webserver1	AlmaLinux 9	192.168.1.21	Web server target for labs
Managed Node 2	webserver2	AlmaLinux 9	192.168.1.22	Application server target for labs

Table 2.1 — Lab environment node specifications

All three nodes should be on the same private subnet (192.168.1.0/24) with no firewall rules between them. In a real lab you can use VirtualBox, VMware Workstation, KVM/libvirt, or cloud VMs (AWS EC2, Azure VM, GCP Compute Engine). The operating systems above are recommendations — Ubuntu for the control node because Ansible is most comfortable on Debian-family systems, and AlmaLinux 9 for managed nodes because AlmaLinux is a free, community-maintained, drop-in replacement for Red Hat Enterprise Linux and is widely used in enterprise environments.

TIP · Replace sample IPs with your actual lab IPs

The IP addresses 192.168.1.10/21/22 are examples. If your lab uses a different subnet (e.g. 10.0.0.0/24) or DHCP-assigned addresses, substitute them everywhere they appear in this manual. The structure of every command and playbook remains identical; only the IP literals change.

MALAYALAM NOTE · Malayalam Teaching Note

Lab-il naammukku 3 machines venam — oru control node (Ubuntu), randu managed nodes (AlmaLinux). VirtualBox, VMware, KVM — eethu use cheyyalum mathi. Subnet same aayirikkenganam (192.168.1.0/24). IP addresses nammle lab-ku thaanmaathram maathram — structure onnmaathriyaayirikkum. AlmaLinux ennu parayunnathu free version of Red Hat Enterprise Linux aanu — enterprise-il nerittondu use aakum.

2.2 Setting Hostnames

Hostname consistency is essential because Ansible playbooks, inventory files, and SSH configurations all reference hostnames. Setting clear, role-based hostnames makes troubleshooting far easier. Run the appropriate command on each node — the Ubuntu syntax uses hostnamectl the same way as AlmaLinux, but the package availability for some commands may differ.

BASH

```
# On the control node (Ubuntu 24.04)
sudo hostnamectl set-hostname ansible-control

# On managed node 1 (AlmaLinux 9)
sudo hostnamectl set-hostname webserver1

# On managed node 2 (AlmaLinux 9)
sudo hostnamectl set-hostname webserver2

# Verify on each node
hostnamectl | head -5
```

EXPECTED OUTPUT

```
Static hostname: ansible-control
       Icon name: computer-vm
         Chassis: vm
      Machine ID: a1b2c3d4...
         Boot ID: e5f6a7b8...
```

2.3 Configuring /etc/hosts for Name Resolution

To allow all three nodes to resolve each other by hostname without relying on a DNS server, add the following entries to the /etc/hosts file on every node. This step is optional but highly recommended for a classroom lab because it simplifies every subsequent command.

BASH

```
# Edit /etc/hosts on all three nodes (use sudo)
sudo nano /etc/hosts

# Add these lines at the end of the file:
192.168.1.10    ansible-control
```

```
192.168.1.21    webserver1
192.168.1.22    webserver2
```

After saving the file, verify name resolution from each node by pinging the other two hostnames. The ping should succeed and the resolved IP should match the one in /etc/hosts.

BASH

```
# From the control node
ping -c 3 webserver1
ping -c 3 webserver2

# From webserver1
ping -c 3 ansible-control
```

EXPECTED OUTPUT

```
PING webserver1 (192.168.1.21) 56(84) bytes of data.
64 bytes from webserver1 (192.168.1.21): icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from webserver1 (192.168.1.21): icmp_seq=2 ttl=64 time=0.385 ms
64 bytes from webserver1 (192.168.1.21): icmp_seq=3 ttl=64 time=0.391 ms

--- webserver1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
```

2.4 Creating the ansible User on Managed Nodes

Best practice is to run Ansible automation through a dedicated, non-personal user account on the managed nodes. This account — we will call it ansible — should have sudo privileges for the specific commands Ansible needs (package installation, service management, file ownership changes) but should not be a personal login account for any human. Using a dedicated account gives you a clear audit trail (every change in /var/log/auth.log shows the ansible user), and it lets you revoke automation access without affecting any human users.

On **both** managed nodes, run the following commands as root or as a user with sudo privileges:

BASH

```
# Create the ansible user with home directory and bash shell
sudo useradd -m -s /bin/bash ansible

# Set a strong password (you will use this only once, for ssh-copy-id)
sudo passwd ansible

# Add ansible to sudoers with NOPASSWD (so Ansible can run sudo without prompting)
echo 'ansible ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/ansible

# Verify
sudo -l -U ansible
```

SECURITY · NOPASSWD sudo is intentional for the lab, narrow it in production

The NOPASSWD: ALL entry allows the ansible user to run any sudo command without typing a password. This is necessary for unattended automation. In a real production environment, you should narrow this to only the specific commands Ansible actually needs — for example, allow only /usr/bin/yum, /usr/bin/systemctl, and /usr/bin/useradd — rather than blanket ALL. We will revisit this in Chapter 4.

MALAYALAM NOTE · Malayalam Teaching Note

Managed nodes-il oru separate user create cheyyunnu — adhinte peru 'ansible'. Aarkum personal aayi use cheyyanda user alla — automation mattondu. Ithu audit-il clear aayirikkum: evide change aakal aanelum 'ansible' user vannu cheythu ennu kanum. NOPASSWD sudo aakanam karanam Ansible interactive aayi password tharaan pattilla. Production-il specific commands mattondu permission tharum (yum, systemctl mattondu) — ALL aakum lab-il matram.

2.5 Verifying SSH Connectivity

From the control node, attempt to SSH into each managed node using the ansible user and the password you set. This step confirms basic connectivity before we move to key-based authentication in the next chapter.

BASH

```
# From ansible-control (as your personal sudo user)
ssh ansible@webserver1
ssh ansible@webserver2

# You should be prompted for the ansible user's password
# Type 'exit' to come back to the control node
```

If the SSH connection fails with *Connection refused*, check that the sshd service is running on the managed node (sudo systemctl status sshd) and that no firewall is blocking port 22. If you see *Permission denied*, double-check the password you set for the ansible user. If you see a long hang followed by timeout, verify the IP addresses and network connectivity with ping.

2.6 Lab Topology Diagram

The diagram below shows the three-node lab topology and the direction of communication. Notice that all SSH connections originate from the control node — the managed nodes never initiate connections back. This is the essence of the push model.

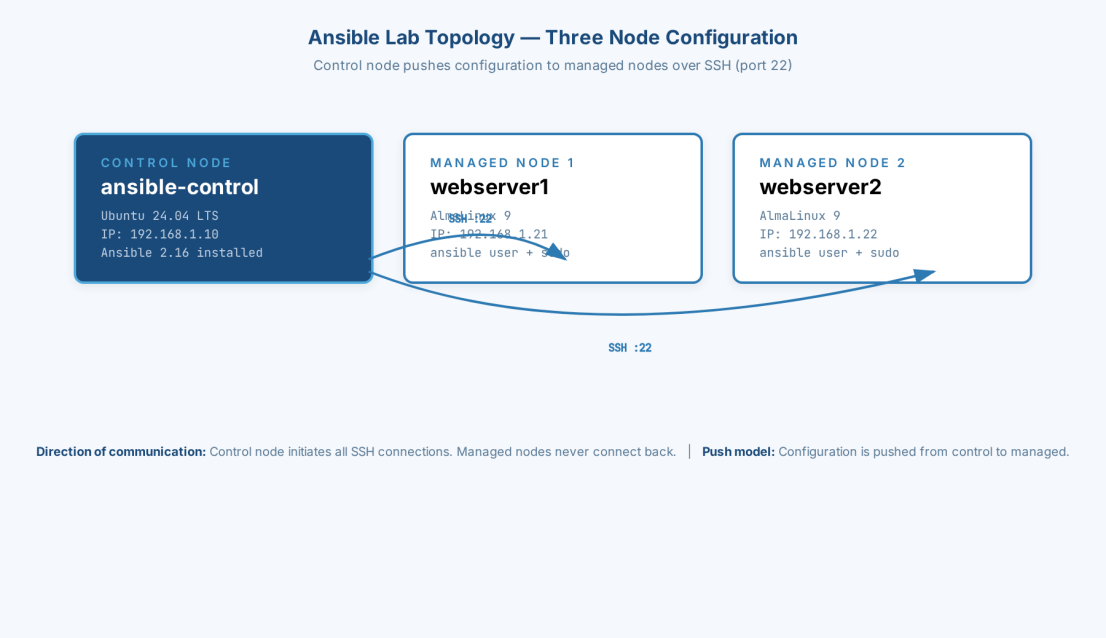


Figure 2.1 — Three-node Ansible lab topology

CHAPTER SUMMARY

The lab consists of three nodes: ansible-control (Ubuntu 24.04, 192.168.1.10), webserver1 (AlmaLinux 9, 192.168.1.21), and webserver2 (AlmaLinux 9, 192.168.1.22). Each node has its hostname set with hostnamectl and resolves the others via /etc/hosts. A dedicated ansible user with NOPASSWD sudo is created on each managed node to provide a clear audit trail and non-personal automation access. SSH connectivity from control to managed nodes is verified before proceeding to Ansible installation.

REVIEW QUESTIONS

1 Why do we recommend using a dedicated ansible user instead of running Ansible as root?

2 What does NOPASSWD: ALL in /etc/sudoers.d/ansible mean? Why is it needed for Ansible?

3 Why is the /etc/hosts file edited on every node instead of using a DNS server?

4 Which node initiates SSH connections in the push model — the control node or the managed node?

5 List two reasons why AlmaLinux is a good choice for the managed nodes in this lab.

6 Which command verifies the hostname has been correctly set?

CHAPTER 3

Installing and Configuring Ansible 2.16

LEARNING OBJECTIVES

- *Install ansible-core 2.16 on Ubuntu 24.04 LTS using apt and pip.*
- *Verify the Ansible installation and version.*
- *Locate and interpret the ansible.cfg configuration file.*
- *Explain the four-level configuration precedence in Ansible.*
- *Identify the key sections of ansible.cfg: [defaults], [privilege_escalation], [ssh_connection].*

3.1 Installation Methods Overview

Ansible can be installed in several ways, each with trade-offs. The two most common methods on Ubuntu are: (1) the apt package manager, which installs the version packaged by Ubuntu (usually an older stable release); and (2) pip, the Python package installer, which installs the latest version directly from the Python Package Index (PyPI). For a classroom lab that needs Ansible 2.16 specifically, we recommend the pip method because it gives you precise version control and is not tied to the distribution release cycle.

Method	Pros	Cons	Best For
apt (Ubuntu package)	Simple, auto-updates with system	Often older version, tied to distro release cycle	Quick start, stable environments
pip (Python PyPI)	Latest version, exact version pinning, works in venv	Manual updates, must use venv on Ubuntu 24.04	Production, version-sensitive labs, this manual
Ansible Community Package	Bundles hundreds of collections	Larger install, includes things you may not need	Users who need many community collections

Table 3.1 — Ansible installation methods compared

3.2 Installing ansible-core 2.16 via pip

On Ubuntu 24.04, Python 3.12 is installed by default but it is managed by the system and protected by PEP 668 (which prevents pip from modifying system packages). The recommended approach is to create a Python virtual environment for Ansible. This isolates Ansible and its dependencies from the system Python and lets you upgrade or downgrade freely without affecting other tools.

BASH

```
# Step 1: Update apt and install Python tooling
sudo apt update
sudo apt install -y python3 python3-pip python3-venv

# Step 2: Create a virtual environment for Ansible
python3 -m venv ~/.ansible-venv

# Step 3: Activate the venv and install ansible-core 2.16
source ~/.ansible-venv/bin/activate
pip install --upgrade pip
pip install 'ansible-core==2.16.*'

# Step 4: Make activation automatic for future shells (optional)
echo 'source ~/.ansible-venv/bin/activate' >> ~/.bashrc
```

After the installation completes, verify the version. The output should show ansible [core 2.16.x] with a recent patch number.

BASH

```
ansible --version
```

EXPECTED OUTPUT

```
ansible [core 2.16.7]
  config file = None
  configured module search path = ['/home/sysadmin/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location =
/home/sysadmin/.ansible-venv/lib/python3.12/site-packages/ansible
  ansible python location = /home/sysadmin/.ansible-venv/bin/python3
  python version = 3.12.3 (main, Apr 10 2024, 05:33:47) [GCC 13.2.0]
  jinja version = 3.1.3
  libyaml = True
```

TIP · Why the version line matters

The ansible --version output reveals several important facts at once: the exact ansible-core version, which Python interpreter is being used (this matters if you have multiple Python installations), where modules are searched for, and whether libyaml (a fast YAML parser) is available. If you ever open a bug report, this output is the first thing maintainers will ask for.

MALAYALAM NOTE · Malayalam Teaching Note

Ubuntu 24.04-il pip directly system-ekku install cheyyan pattilla (PEP 668 karanam). Aakayirunnu nammal virtual environment (venv) use cheyyunnu. Ithu oru separate folder-aayittu ansible-um adhinte dependencies-um store aakkum — system Python-ine kondu conflict aakilla. Production-il venv use cheyyunnathu best practice aanu. ansible-core==2.16. enna vachaal latest 2.16 patch kittum.*

3.3 The ansible.cfg Configuration File

Ansible reads its runtime configuration from a file named ansible.cfg. This file controls global defaults like the inventory location, the SSH user, the number of parallel forks, the privilege escalation method, and many more settings. You can run Ansible without an ansible.cfg (it has sensible defaults), but for any non-trivial project you will want one. The file uses INI syntax with sections in square brackets.

3.4 Configuration Precedence (Four Levels)

Ansible looks for its configuration in four places, in priority order. The first one found is used; the rest are ignored. This precedence is the source of many subtle bugs — if Ansible is behaving differently than you expect, the first thing to check is which ansible.cfg is actually being read.

Priority	Source	How to Override
1 (highest)	Environment variable ANSIBLE_CONFIG	export ANSIBLE_CONFIG=/path/to/ansible.cfg
2	ansible.cfg in current working directory	cd into project folder containing ansible.cfg
3	~/.ansible.cfg in user home directory	Create personal defaults file
4 (lowest)	/etc/ansible/ansible.cfg system-wide	Edit as root, affects all users

Table 3.2 — ansible.cfg precedence (highest first)

To see which configuration file Ansible is currently using, run ansible-config dump or check the second line of ansible --version output. The config file = None line we saw earlier means Ansible is using built-in defaults because no ansible.cfg was found in any of the four locations.

3.5 Creating a Project-Level ansible.cfg

For our lab we will create a project-level ansible.cfg in the working directory ~/ansible-lab. This is the recommended approach for classroom work because it is self-contained, easy to inspect, and travels with the project if you copy or version-control it.

BASH

```
# Create the project directory
mkdir -p ~/ansible-lab
cd ~/ansible-lab

# Create ansible.cfg
nano ansible.cfg
```

Paste the following content into the file:

INI

```
[defaults]
; Where to find the inventory file
inventory = ./inventory

; Default SSH user
remote_user = ansible

; Number of parallel hosts to manage at once
forks = 5

; Disable host key checking (only for lab use!)
host_key_checking = False

; Use the YAML callback for nicer output
```

```
stdout_callback = yaml

; Where to look for roles
roles_path = ./roles

[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False

[ssh_connection]
; Use ControlPersist for faster repeated SSH connections
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
pipelining = True
```

Let us walk through the most important settings. inventory tells Ansible where to find the list of hosts to manage. remote_user is the SSH user Ansible will log in as (we created this user in Chapter 2). forks controls parallelism — with 5 forks Ansible will work on up to 5 hosts simultaneously, which is plenty for our 2-node lab. host_key_checking = False disables the SSH host-key verification prompt — this is a security trade-off for classroom convenience. **Never disable host key checking in production.**

The [privilege_escalation] section tells Ansible to automatically use sudo to become root on managed hosts. Because we configured NOPASSWD sudo for the ansible user in Chapter 2, this will work without prompting for a password. The [ssh_connection] section enables SSH multiplexing, which dramatically speeds up playbook runs by reusing a single SSH connection for multiple tasks.

SECURITY · host_key_checking = False is dangerous in production

Disabling host key checking means Ansible will accept any SSH server identity without verifying it. In a classroom lab this is fine, but in production it opens the door to man-in-the-middle attacks. The proper production approach is to pre-populate ~/.ssh/known_hosts with the managed nodes' public keys (using ssh-keyscan) and then leave host_key_checking = True.

MALAYALAM NOTE · Malayalam Teaching Note

ansible.cfg file-il nammal 3 sections use cheyyunnu: [defaults] general settings-nokku, [privilege_escalation] sudo-veku vendi, [ssh_connection] SSH speed-up vendi. host_key_checking False aakkiyathu lab-il matram — production-il aakeem cheyyanda. ssh_args ControlMaster-vekkal SSH connection reuse aakkum, adhu valiya speed improvement tharum. forks = 5 aanel 5 hosts-ekku parallel aayi work cheyyum.

3.6 Verifying the Configuration

After saving ansible.cfg, verify that Ansible now picks it up. The config file line should now show the path to your file instead of None.

BASH

```
cd ~/ansible-lab
ansible --version | head -2
```

EXPECTED OUTPUT

```
ansible [core 2.16.7]
  config file = /home/sysadmin/ansible-lab/ansible.cfg
```

You can also dump the full effective configuration with ansible-config dump. This is invaluable for debugging — it shows the resolved value of every setting, including which file it came from.

BASH

```
ansible-config dump | grep -E 'HOST_KEY|REMOTE_USER|INVENTORY'
```

EXPECTED OUTPUT

```
HOST_KEY_CHECKING(/home/sysadmin/ansible-lab/ansible.cfg) = False
INVENTORY_ANYHOSTS(/home/sysadmin/ansible-lab/ansible.cfg) = ./inventory
REMOTE_USER(/home/sysadmin/ansible-lab/ansible.cfg) = ansible
```

CHAPTER SUMMARY

Install ansible-core 2.16 in a Python virtual environment to avoid conflicts with system packages. The ansible.cfg file controls runtime behaviour through three main sections: [defaults] for general settings, [privilege_escalation] for sudo, and [ssh_connection] for SSH optimisation. Configuration precedence is: ANSIBLE_CONFIG env var > ansible.cfg in current directory > ~/.ansible.cfg > /etc/ansible/ansible.cfg. Always verify the active config with ansible --version before debugging unexpected behaviour.

REVIEW QUESTIONS

1. Why do we recommend using a virtual environment for Ansible on Ubuntu 24.04?
2. What is the difference between ansible-core and the ansible community package?
3. List the four levels of ansible.cfg precedence, in priority order.
4. What does the forks setting control? What is a sensible value for a 2-node lab?
5. Why is host_key_checking = False acceptable in a classroom lab but dangerous in production?
6. Which ansible.cfg section configures sudo privilege escalation?
7. Which command shows the full effective Ansible configuration, including source file?

CHAPTER 4

SSH, Sudo Privilege Escalation, and Security

LEARNING OBJECTIVES

- *Generate an SSH key pair using ed25519 (the modern recommendation).*
- *Use ssh-copy-id to install the public key on managed nodes.*
- *Verify SSH key-based login works without a password.*
- *Configure sudoers files safely using visudo and /etc/sudoers.d/.*
- *Explain Ansible's become/become_user directives and their security implications.*
- *Apply best practices for credential handling in automation.*

4.1 Why SSH Key Authentication?

Ansible connects to managed nodes over SSH. By default SSH accepts either password or key-based authentication, but for automation, key-based authentication is strongly preferred for three reasons. First, it is non-interactive — Ansible can connect to thousands of hosts without a human typing a password each time. Second, it is more secure than passwords because private keys are 256-bit (ed25519) or 3072-bit (RSA) secrets that cannot be brute-forced. Third, it is auditable — each key pair is unique to a control node, so log entries show exactly which control node initiated each connection.

MALAYALAM NOTE · Malayalam Teaching Note

SSH key authentication aakumbol password onnum tharaanamilla — Ansible automatic aayi connect aakkum. Key-based authentication password-ine kaalum safe aanu karanam ed25519 key 256-bit aanu, brute-force cheyyan impossible. Audit aakumbol specific control node-il ninnu vannathaayi kaanum.

4.2 Generating an SSH Key Pair

On the control node, generate a key pair using the ed25519 algorithm. Ed25519 is the modern recommendation from OpenSSH because it is faster, produces shorter keys, and is resistant to many attacks that affect RSA. Avoid the older RSA algorithm unless you have a specific compatibility requirement.

BASH

```
# On the control node, as your personal user (not root)
ssh-keygen -t ed25519 -C 'ansible-control' -f ~/.ssh/ansible_ed25519

# You will be prompted for a passphrase. For a lab you can leave it empty
# (just press Enter twice). For production, ALWAYS use a strong passphrase.
```

EXPECTED OUTPUT

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sysadmin/.ssh/ansible_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sysadmin/.ssh/ansible_ed25519
Your public key has been saved in /home/sysadmin/.ssh/ansible_ed25519.pub
The key fingerprint is:
SHA256:AbCdEf... ansible-control
The key's randomart image is:
+--[ED25519 256]--+
|      ...        |
+----[SHA256]-----+
```

This command produces two files in ~/.ssh/: the private key ansible_ed25519 (which must be kept secret) and the public key ansible_ed25519.pub (which is safe to share and install on remote hosts). Protect the private key with strict file permissions — if other users can read it, SSH will refuse to use it.

BASH

```
# Set strict permissions on the private key
chmod 700 ~/.ssh
chmod 600 ~/.ssh/ansible_ed25519
chmod 644 ~/.ssh/ansible_ed25519.pub

# Verify
ls -l ~/.ssh/ansible_ed25519*
```

EXPECTED OUTPUT

```
-rw------- 1 sysadmin sysadmin  411 Oct 24 10:15 /home/sysadmin/.ssh/ansible_ed25519
-rw-r--r-- 1 sysadmin sysadmin  103 Oct 24 10:15 /home/sysadmin/.ssh/ansible_ed25519.pub
```

4.3 Installing the Public Key on Managed Nodes

The public key must be appended to the ~/.ssh/authorized_keys file of the ansible user on each managed node. The easiest way to do this is with the ssh-copy-id command, which handles all the file permission details automatically. You will be prompted for the ansible user password one last time; after this step, password authentication is no longer needed.

BASH

```
# Copy the public key to both managed nodes
ssh-copy-id -i ~/.ssh/ansible_ed25519.pub ansible@webserver1
ssh-copy-id -i ~/.ssh/ansible_ed25519.pub ansible@webserver2
```

EXPECTED OUTPUT

```
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/home/sysadmin/.ssh/ansible_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is
to install the new keys
ansible@webserver1's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh -i '/home/sysadmin/.ssh/ansible_ed25519'
ansible@webserver1"
```

```
and check to make sure that only the key(s) you wanted were added.
```

Test that key-based login now works without a password. If you are prompted for a password, something is wrong with the key installation — check the file permissions on ~/.ssh/authorized_keys on the managed node (it must be 600 and owned by the ansible user).

BASH

```
# This should log in WITHOUT a password prompt
ssh -i ~/.ssh/ansible_ed25519 ansible@webserver1 'hostname; whoami'
ssh -i ~/.ssh/ansible_ed25519 ansible@webserver2 'hostname; whoami'
```

EXPECTED OUTPUT

```
webserver1
ansible
webserver2
ansible
```

4.4 Configuring SSH to Use the Key Automatically

Typing -i ~/.ssh/ansible_ed25519 on every command is tedious. The standard solution is to add an entry to ~/.ssh/config that tells SSH which key to use for which host. Create or edit the file as shown below.

BASH

```
nano ~/.ssh/config

# Add the following content:
Host webserver1 webserver2
    User ansible
    IdentityFile ~/.ssh/ansible_ed25519
    IdentitiesOnly yes

# Set permissions (SSH refuses to read config that is group/world readable)
chmod 600 ~/.ssh/config
```

Now you can simply type ssh webserver1 and SSH will automatically use the correct key and the correct user. IdentitiesOnly yes tells SSH to use only the specified identity file, which avoids the situation where SSH tries every key in your agent and triggers too-many-failures lockouts on the remote host.

4.5 Sudoers Configuration (Recap and Production Hardening)

In Chapter 2 we created a permissive /etc/sudoers.d/ansible file with NOPASSWD: ALL. For a classroom lab this is fine, but for production you should narrow the sudo permissions to only what Ansible actually needs. The safe way to edit sudoers is with visudo, which validates the syntax before saving — a syntax error in sudoers can lock you out of root entirely.

BASH

```
# Always use visudo to edit sudoers files!
sudo visudo -f /etc/sudoers.d/ansible

# Production-safe example: only allow Ansible to run specific commands
ansible ALL=(ALL) NOPASSWD: /usr/bin/yum, /usr/bin/dnf, /usr/bin/rpm, \
    /usr/bin/systemctl, /usr/bin/useradd, /usr/bin/usermod, \
    /usr/bin/groupadd, /usr/bin/mkdir, /usr/bin/chown, /usr/bin/chmod, \
```

```
    /usr/bin/firewall-cmd, /usr/bin/sed, /usr/bin/cp, /usr/bin/mv
```

SECURITY · NEVER edit /etc/sudoers directly with a text editor

Always use visudo to edit either the main sudoers file or files in /etc/sudoers.d/. visudo performs a syntax check before saving. A broken sudoers file prevents sudo from working at all, which can permanently lock you out of root access — the only fix is single-user mode boot from console.

4.6 Ansible become Directives

Ansible can escalate privileges on managed hosts using the become family of directives. These map to sudo by default but can also use su, pbrun, pfexec, or doas. The directives can be set at the playbook level, the play level, the task level, or globally in ansible.cfg — the most specific setting wins.

Directive	Level	Purpose
become: true	play / task	Enable privilege escalation for this play or task
become_user: root	play / task	Which user to escalate to (default: root)
become_method: sudo	play / task	Which escalation tool to use (default: sudo)
become_ask_pass: false	global	Whether to prompt for sudo password (default: false)
--become / -b	CLI flag	Enable become for ad-hoc commands
--become-user=USER	CLI flag	Override become_user from CLI

Table 4.1 — Ansible become privilege escalation directives

Because we set become = True in our ansible.cfg in Chapter 3, every task will automatically use sudo. You can override this per-task with become: false when you specifically want a task to run as the ansible user (for example, when installing a Python package into the ansible user's home directory).

YAML

```yaml
# Example playbook showing become at different levels
---
- name: Demonstrate become directives
  hosts: webservers
  become: true                 # Default for all tasks in this play
  tasks:
    - name: Install nginx (as root via sudo)
      ansible.builtin.yum:
        name: nginx
        state: present

    - name: Check ansible user's home directory (as ansible user, no sudo)
      ansible.builtin.command: ls -la ~/
      become: false            # Override for this task only
      register: home_listing

    - name: Show the listing
      ansible.builtin.debug:
```

```
    var: home_listing.stdout_lines
```

4.7 Secure Credential Handling Best Practices

- **Never commit private keys or passwords to version control.** Add them to .gitignore. If you accidentally commit a key, rotate it immediately — assume it is compromised.
- **Use Ansible Vault** (Chapter 14) to encrypt any sensitive data that must travel with your playbooks, such as database passwords or API tokens.
- **Restrict sudo permissions** to only the commands Ansible actually needs. Avoid NOPASSWD: ALL in production.
- **Use a passphrase on SSH keys** in production, and load the passphrase into ssh-agent once per session. This protects the key even if the file is stolen.
- **Rotate keys regularly** — at minimum annually, and immediately when an administrator leaves the team.
- **Audit sudo usage** by reviewing /var/log/auth.log (Ubuntu) or /var/log/secure (AlmaLinux) on managed nodes for the ansible user.

MALAYALAM NOTE · Malayalam Teaching Note

Sudoers file edit cheyyumbol visudo use cheyyanam — direct edit cheyynaal syntax error vannaal sudo ozhinjaal nammal lock aakkum. SSH key-ntu passphrase vekkuka production-il — key stolen aakum aakumbolum password vannu tharanam. Ansible Vault use cheyyuka passwords-um API keys-um encrypt cheyyaan. Git-ilekk private keys passwords ittaruthu — accidental commit aakumbol adhine rotate cheyyuka immediately.

CHAPTER SUMMARY

SSH key-based authentication (preferably ed25519) is the foundation of secure Ansible automation. Generate keys on the control node, install the public key with ssh-copy-id, configure ~/.ssh/config for convenience, and use visudo to safely manage sudoers files. Ansible's become directives control privilege escalation per-play and per-task. In production, narrow NOPASSWD to specific commands, use passphrases on keys with ssh-agent, and encrypt sensitive data with Ansible Vault.

REVIEW QUESTIONS

1. Why is ed25519 preferred over RSA for SSH keys?
2. What permissions should the private key file have? Why does SSH refuse to use it otherwise?
3. Explain the purpose of IdentitiesOnly yes in ~/.ssh/config.
4. Why must you always use visudo to edit sudoers files?
5. What does become: false do when placed on a single task within a play that has become: true?
6. List three best practices for handling credentials in Ansible automation.
7. Where would you look on a managed node to audit sudo activity by the ansible user?

CHAPTER 5

Inventory Management

LEARNING OBJECTIVES

- *Create a static inventory file in both INI and YAML formats.*
- *Organise hosts into groups and groups of groups (children).*
- *Set host variables and group variables in the inventory.*
- *Use common inventory parameters: ansible_host, ansible_user, ansible_port, ansible_ssh_private_key_file.*
- *Verify and inspect the inventory with ansible-inventory --graph and --list.*

5.1 What is an Inventory?

An inventory is a list of hosts that Ansible manages, organised into groups for convenience. The inventory can be a static text file (the simplest case), a YAML file (more structured), or a dynamic script that fetches hosts from a cloud provider (advanced, covered later). When you run an Ansible command, you point it at a group or a host from the inventory, and Ansible connects to those machines in parallel.

MALAYALAM NOTE · Malayalam Teaching Note

Inventory ennu parayunnathu Ansible manage cheyyunna servers-le list aanu. Adhu oru text file aakum, YAML aakum, dynamic script aakum (cloud providers-nokku). Groups aayi organise cheyyum — for example webservers, dbservers. Oru command run cheyyumbol oru group-ku nokki run cheyyum, Ansible parallel aayi ellaa hosts-umekku connect aakkum.

5.2 The INI Inventory Format (Most Common)

The INI format is the most widely used for static inventories because it is simple and human-readable. Group names are written in square brackets, and hosts are listed below each group. A host can belong to multiple groups. Create the file ~/ansible-lab/inventory with the following content.

INI

```
# ~/ansible-lab/inventory

# Ungrouped hosts go at the top (we have none in this lab)

# Group: webservers
[webservers]
webserver1 ansible_host=192.168.1.21
webserver2 ansible_host=192.168.1.22

# Group: all_alma (every AlmaLinux host)
[all_alma]
```

```
webserver1
webserver2

# Group of groups: datacentre contains webservers and (future) dbservers
[datacentre:children]
webservers

# Group variables for webservers
[webservers:vars]
ansible_user=ansible
ansible_ssh_private_key_file=/home/sysadmin/.ssh/ansible_ed25519
ansible_python_interpreter=/usr/bin/python3
```

Notice several key features of this inventory. First, ansible_host overrides the hostname for the SSH connection — this lets you use a friendly name like webserver1 in playbooks while connecting to the IP address. Second, groups can have a :vars subsection that applies variables to every host in the group. Third, [datacentre:children] defines a group of groups — any host in webservers is automatically in datacentre.

5.3 Common Inventory Parameters

Parameter	Default	Purpose
ansible_host	hostname from inventory	IP or DNS name to SSH to
ansible_user	current user	SSH user (e.g. ansible)
ansible_port	22	SSH port (change for non-standard SSH)
ansible_ssh_private_key_file	~/.ssh/id_*	Path to private key
ansible_python_interpreter	auto-detected	Path to Python on managed host
ansible_become	from ansible.cfg	Whether to use sudo
ansible_become_user	root	Which user to escalate to
ansible_shell_type	sh	Shell type (sh, bash, csh, fish)
ansible_ssh_common_args	—	Extra SSH args (e.g. for ProxyJump)

Table 5.1 — Common inventory parameters

5.4 The YAML Inventory Format

Ansible 2.x also supports YAML inventory files (with the .yml or .yaml extension). YAML inventory is more verbose but allows richer structure and easier programmatic generation. Below is the same inventory expressed in YAML.

YAML

```
# ~/ansible-lab/inventory.yml
---
all:
  children:
```

```
    webservers:
      vars:
        ansible_user: ansible
        ansible_ssh_private_key_file: /home/sysadmin/.ssh/ansible_ed25519
        ansible_python_interpreter: /usr/bin/python3
      hosts:
        webserver1:
          ansible_host: 192.168.1.21
        webserver2:
          ansible_host: 192.168.1.22
    all_alma:
      hosts:
        webserver1:
        webserver2:
    datacentre:
      children:
        webservers:
```

Choose whichever format you find more readable. The INI format is more common in community examples and tutorials, so this manual will use INI for most examples. YAML inventory is preferred when you generate inventory programmatically (e.g. from Terraform output) because YAML is easier to emit from code.

5.5 Host Variables vs Group Variables

Variables can be defined at three levels: per-host (in the inventory line itself or in host_vars/), per-group (in [group:vars] or group_vars/), and globally (in group_vars/all.yml). The rule is: **more specific wins**. A host variable overrides a group variable, which overrides an all-group variable.

TEXT

```
# Directory layout for advanced variable management
ansible-lab/
  ansible.cfg
  inventory
  group_vars/
    all.yml            # Variables for ALL hosts
    webservers.yml     # Variables for the webservers group
  host_vars/
    webserver1.yml     # Variables for webserver1 only
    webserver2.yml     # Variables for webserver2 only
```

The group_vars/ and host_vars/ directories are automatically picked up by Ansible when they sit next to the inventory file or playbook. Each YAML file in these directories contains variables for the named group or host. This is the recommended approach for any non-trivial project because it keeps the inventory file clean and groups variables logically.

YAML

```
# ~/ansible-lab/group_vars/all.yml
---
ansible_user: ansible
ansible_python_interpreter: /usr/bin/python3

# ~/ansible-lab/group_vars/webservers.yml
---
nginx_version: '1.25.3'
```

```
doc_root: /var/www/html

# ~/ansible-lab/host_vars/webserver1.yml
---
server_role: primary
max_connections: 200
```

5.6 Verifying the Inventory

The ansible-inventory command is invaluable for inspecting and debugging inventories. It can show a tree view, a flat list, or a JSON dump. Always run it after creating or modifying an inventory to confirm Ansible sees what you expect.

BASH

```
cd ~/ansible-lab

# Show the inventory as a tree
ansible-inventory --graph
```

EXPECTED OUTPUT

```
@all:
  |--@datacentre:
  |  |--@webservers:
  |  |  |--webserver1
  |  |  |--webserver2
  |--@all_alma:
  |  |--webserver1
  |  |--webserver2
  |--@ungrouped:
```

BASH

```
# Show all variables for a specific host (as JSON)
ansible-inventory --host webserver1
```

EXPECTED OUTPUT

```
{
    "ansible_host": "192.168.1.21",
    "ansible_python_interpreter": "/usr/bin/python3",
    "ansible_ssh_private_key_file": "/home/sysadmin/.ssh/ansible_ed25519",
    "ansible_user": "ansible"
}
```

5.7 The Default Inventory Locations

If you do not specify an inventory file with -i or in ansible.cfg, Ansible looks in two default locations: /etc/ansible/hosts (system-wide) and ~/.ansible_hosts or ~/.config/ansible/hosts (per-user). For classroom work we strongly recommend using a project-level inventory configured in ansible.cfg, as we did in Chapter 3. This keeps everything self-contained and reproducible.

CHAPTER SUMMARY

An inventory is a list of hosts organised into groups that Ansible manages. The INI format is the most common for static inventories; YAML is preferred when generating inventory programmatically. Hosts can belong to multiple groups, and groups can contain other groups (children). Variables can be set per-host, per-group, or globally, with more specific values overriding more general ones. Use ansible-inventory --graph and --host to verify your inventory after every change.

REVIEW QUESTIONS

1 Write an INI inventory snippet that defines a group dbservers containing one host db1 with IP 192.168.1.30.

2 What is the difference between [webservers] and [webservers:vars] in an INI inventory?

3 Explain the difference between group_vars/ and host_vars/ directories.

4 Which command shows the resolved variables for a single host as JSON?

5 Why might you set ansible_python_interpreter explicitly in the inventory?

6 What does [datacentre:children] mean in an INI inventory?

CHAPTER 6

Ad-Hoc Commands

LEARNING OBJECTIVES

- *Understand the anatomy of an Ansible ad-hoc command.*
- *Use common modules via ad-hoc: ping, command, shell, copy, yum, service, user, file, setup.*
- *Apply parallelism with --forks, host filtering with --limit, and verbosity with -v to -vvvv.*
- *Use -b for sudo privilege escalation in ad-hoc commands.*
- *Recognise common mistakes in ad-hoc command usage.*

6.1 What is an Ad-Hoc Command?

An ad-hoc command is a one-shot Ansible invocation from the command line, used to perform a quick task without writing a full playbook. Ad-hoc commands are perfect for tasks like rebooting all servers, checking disk usage, copying a file to many hosts, or restarting a service. For repeatable, complex, or version-controlled work you should write a playbook instead, but for one-off operations ad-hoc is the right tool.

6.2 Anatomy of an Ad-Hoc Command

Every ad-hoc command follows the same pattern: ansible -m -a . The host pattern selects which hosts to target (a group name, a host name, or a pattern like webservers[0]). The -m flag specifies which module to run. The -a flag passes arguments to the module. Optional flags add sudo, parallelism, verbosity, and more.

BASH

```
ansible webservers -m ansible.builtin.ping
```

This is the simplest possible ad-hoc command. It targets the webservers group from the inventory, runs the ping module, and takes no arguments. The ping module does not actually send ICMP echo requests — it tests that Ansible can SSH into the host and run Python. A successful response is pong.

EXPECTED OUTPUT

```
webserver1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
webserver2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

6.3 Common Ad-Hoc Patterns

Pattern 1: Run a command on remote hosts

BASH

```
# Check uptime on all webservers
ansible webservers -m ansible.builtin.command -a 'uptime'

# Check free memory
ansible webservers -m ansible.builtin.command -a 'free -h'

# Check disk usage
ansible webservers -m ansible.builtin.command -a 'df -h'
```

Pattern 2: Install a package (idempotent)

BASH

```
# Install nginx on webservers using yum module (AlmaLinux)
ansible webservers -m ansible.builtin.yum -a 'name=nginx state=present' -b

# Install httpd on webservers using apt module (Ubuntu)
# ansible webservers -m ansible.builtin.apt -a 'name=apache2 state=present update_cache=yes'
-b
```

Note the -b flag, which enables sudo. Without it the yum/apt module would fail because the ansible user does not have permission to install packages. The state=present argument means *ensure the package is installed* — this is idempotent, so re-running it does nothing if nginx is already there.

Pattern 3: Start and enable a service

BASH

```
# Start nginx now and enable it to start on boot
ansible webservers -m ansible.builtin.service -a 'name=nginx state=started enabled=yes' -b
```

Pattern 4: Create a user

BASH

```
# Create a new user with home directory and bash shell
ansible webservers -m ansible.builtin.user -a 'name=appuser shell=/bin/bash groups=wheel
append=yes' -b
```

Pattern 5: Copy a file to remote hosts

BASH

```
# Copy a local file to remote hosts with specific ownership and permissions
ansible webservers -m ansible.builtin.copy -a 'src=/etc/motd dest=/etc/motd owner=root
group=root mode=0644 backup=yes' -b
```

Pattern 6: Gather facts (system information)

BASH

```
# Gather facts about all webservers (long output!)
ansible webservers -m ansible.builtin.setup

# Filter for specific fact (IPv4 address)
ansible webservers -m ansible.builtin.setup -a 'filter=ansible_default_ipv4'
```

6.4 Useful Ad-Hoc Flags

Flag	Purpose	Example
-b, --become	Run with sudo	ansible webservers -b -m yum -a "name=nginx state=present"
-K, --ask-become-pass	Prompt for sudo password	Use when sudo requires a password
-f N, --forks=N	Number of parallel hosts	-f 10 to run on 10 hosts at once
--limit HOST	Restrict to subset of hosts	--limit webserver1
-v / -vv / -vvv / -vvvv	Increase verbosity	-vvv shows full SSH output
-i INVENTORY	Use a specific inventory file	-i ./production-inventory
-u USER	Override SSH user	-u root to log in as root directly
--check	Dry run (no changes)	--check to preview what would change
--diff	Show diffs of changes	Combine with --check for safe preview

Table 6.1 — Common ad-hoc command flags

6.5 Verbosity Levels Explained

Ansible has four verbosity levels, each adding more detail to the output. Knowing when to use each is a critical troubleshooting skill.

- **-v** — Shows the result of each task (verbose). Use for normal debugging.
- **-vv** — Adds task path and module arguments. Use when a module is not behaving as expected.
- **-vvv** — Adds SSH connection details. Use when SSH connectivity is the problem.
- **-vvvv** — Adds SSH plug-in internals. Use for very deep SSH-level debugging.

BASH

```
# Example: debug a failing connection with maximum verbosity
ansible webserver1 -m ansible.builtin.ping -vvvv
```

6.6 Common Mistakes with Ad-Hoc Commands

Mistake	Why It Fails	Fix
Forgetting -b for privileged tasks	ansible user lacks permission to install packages	Add -b (or set become=True in ansible.cfg)
Using shell module for everything	Shell is not idempotent by default; command injection risk	Use specific modules (yum, service, user) when available
Not quoting -a arguments	Shell parsing breaks on spaces	Wrap -a argument in single quotes

Mistake	Why It Fails	Fix
Running on wrong inventory	Default inventory used instead of project inventory	Use cd to project dir or -i flag
Ignoring yellow warnings	Warnings indicate deprecated syntax or potential issues	Read and address every warning

Table 6.2 — Common ad-hoc command mistakes

MALAYALAM NOTE · Malayalam Teaching Note

Ad-hoc command ennu parayunnathu oru quick one-shot task aanu — playbook ezhuthanda avashyam illa. Pattern: ansible -m -a . -b vecho sudo-vekanam privileged tasks-ku. -v, -vv, -vvv, -vvvv verbosity koodutharam aakkum — troubleshooting-ku sahayikkum. Specific modules prefer cheyyuka shell-ine kaalum — yum, service, user, etc. — aakayal idempotent aayirikkum.

CHAPTER SUMMARY

Ad-hoc commands perform one-shot tasks via the pattern ansible -m -a . Use them for quick operations like checking uptime, installing a package, restarting a service, or copying a file. The -b flag enables sudo, --limit restricts targets, --check performs a dry run, and -v through -vvvv increase verbosity for debugging. Prefer specific modules (yum, service, user) over the generic shell module for idempotency and safety.

REVIEW QUESTIONS

1. Write an ad-hoc command to install the package vim on all webservers using sudo.
2. What is the difference between command and shell modules? When would you use each?
3. Which flag performs a dry run? Why is it useful?
4. Explain the four verbosity levels and when you would use each.
5. Why should you prefer the user module over shell -a 'useradd ...'?
6. Write an ad-hoc command to check disk usage on webserver1 only.

CHAPTER 7

Ansible Modules — Deep Dive

LEARNING OBJECTIVES

- *Categorise Ansible modules by function: system, package, file, command, network, cloud.*
- *Use the most important modules with correct syntax: user, group, service, systemd, yum, apt, file, copy, template, lineinfile, fetch, synchronize, command, shell, raw, script, firewalld, git.*
- *Understand the difference between command, shell, raw, and script modules.*
- *Apply idempotency principles when choosing modules.*

7.1 What is a Module?

A module is a small, reusable piece of Python code that Ansible ships to the managed node and executes there. Modules take arguments, perform a specific action (install a package, create a user, modify a file), and return a JSON result. Ansible ships with hundreds of built-in modules in the ansible.builtin collection, and the wider Ansible Galaxy community hosts thousands more in additional collections like community.general, community.docker, and so on.

The most useful command for learning any module is ansible-doc. It shows the module's documentation, parameters, and examples offline — no internet needed.

BASH

```
# Show documentation for the yum module
ansible-doc ansible.builtin.yum

# Show a short snippet (synopsis + examples)
ansible-doc -s ansible.builtin.copy

# List all available modules
ansible-doc -l | head -20
```

7.2 System Modules

user — Create and manage users

BASH

```
# Create user with bash shell, home dir, and add to wheel group
ansible webservers -m ansible.builtin.user -a 'name=appuser shell=/bin/bash groups=wheel
append=yes' -b

# Create user with a password hash (use python -c to generate the hash)
ansible webservers -m ansible.builtin.user -a 'name=appuser password=$6$mysecret...' -b

# Remove a user and their home directory
```

```
ansible webservers -m ansible.builtin.user -a 'name=olduser state=absent remove=yes' -b
```

group — Create and manage groups

BASH

```
# Create a group
ansible webservers -m ansible.builtin.group -a 'name=developers state=present' -b

# Remove a group
ansible webservers -m ansible.builtin.group -a 'name=oldgroup state=absent' -b
```

service / systemd — Manage services

BASH

```
# Start nginx now and enable on boot
ansible webservers -m ansible.builtin.service -a 'name=nginx state=started enabled=yes' -b

# Restart a service (idempotent: only restarts if config changed; use handlers for that)
ansible webservers -m ansible.builtin.service -a 'name=nginx state=restarted' -b

# Stop and disable
ansible webservers -m ansible.builtin.service -a 'name=nginx state=stopped enabled=no' -b

# Reload (re-reads config without dropping connections)
ansible webservers -m ansible.builtin.service -a 'name=nginx state=reloaded' -b
```

hostname — Set the system hostname

BASH

```
# Set hostname (persists across reboots on systemd systems)
ansible webserver1 -m ansible.builtin.hostname -a 'name=webserver1.example.com' -b
```

reboot — Reboot and wait for the host to come back

BASH

```
# Reboot all webservers and wait for them to come back online
ansible webservers -m ansible.builtin.reboot -b
```

7.3 Package Modules

yum / dnf — RPM package management (AlmaLinux, RHEL, Rocky)

BASH

```
# Install a single package
ansible webservers -m ansible.builtin.yum -a 'name=nginx state=present' -b

# Install multiple packages at once (list syntax)
ansible webservers -m ansible.builtin.yum -a 'name=[nginx,git,vim] state=present' -b

# Install the latest version
ansible webservers -m ansible.builtin.yum -a 'name=nginx state=latest' -b

# Remove a package
ansible webservers -m ansible.builtin.yum -a 'name=nginx state=absent' -b
```

apt — Debian package management (Ubuntu, Debian)

BASH

```
# Install a package and update cache
ansible webservers -m ansible.builtin.apt -a 'name=nginx state=present update_cache=yes' -b

# Remove a package and its config files
ansible webservers -m ansible.builtin.apt -a 'name=nginx state=absent purge=yes' -b
```

pip — Python package management

BASH

```
# Install a Python package
ansible webservers -m ansible.builtin.pip -a 'name=requests' -b
```

7.4 File Modules

file — Set file properties (ownership, permissions, state)

BASH

```
# Create a directory with specific owner, group, and mode
ansible webservers -m ansible.builtin.file -a 'path=/var/www/example state=directory
owner=nginx group=nginx mode=0755' -b

# Create an empty file
ansible webservers -m ansible.builtin.file -a 'path=/var/log/myapp.log state=touch
owner=nginx mode=0644' -b

# Delete a file or directory
ansible webservers -m ansible.builtin.file -a 'path=/tmp/oldstuff state=absent' -b

# Create a symlink
ansible webservers -m ansible.builtin.file -a 'src=/etc/nginx/nginx.conf
dest=/etc/nginx/conf.d/default.conf state=link' -b
```

copy — Copy a file from control node to managed hosts

BASH

```
# Copy a local file to remote with ownership and permissions
ansible webservers -m ansible.builtin.copy -a 'src=./files/motd dest=/etc/motd owner=root
group=root mode=0644 backup=yes' -b

# Copy inline content (no source file needed)
ansible webservers -m ansible.builtin.copy -a 'content="Hello from Ansible"
dest=/tmp/hello.txt mode=0644' -b
```

lineinfile — Manage a single line in a file (idempotent)

BASH

```
# Ensure a line is present (replaces if it matches a regex)
ansible webservers -m ansible.builtin.lineinfile -a 'path=/etc/selinux/config
regexp="^SELINUX=" line="SELINUX=disabled"' -b

# Ensure a line is absent (comment out a setting)
```

```
ansible webservers -m ansible.builtin.lineinfile -a 'path=/etc/ssh/sshd_config
regexp="^PermitRootLogin" state=absent' -b
```

blockinfile — Manage a block of lines

BASH

```
# Add a managed block of lines (Ansible adds marker comments)
ansible webservers -m ansible.builtin.blockinfile -a '
  path=/etc/ssh/sshd_config
  marker="{mark} ANSIBLE MANAGED BLOCK"
  block="|\n    ClientAliveInterval 300\n    ClientAliveCountMax 2"
' -b
```

fetch — Copy a file from managed host to control node

BASH

```
# Pull a log file from each webserver back to the control node
ansible webservers -m ansible.builtin.fetch -a 'src=/var/log/nginx/access.log dest=./logs/
flat=yes' -b
```

synchronize — rsync wrapper for efficient file sync

BASH

```
# Sync an entire directory tree to remote hosts (requires rsync on both ends)
ansible webservers -m ansible.posix.synchronize -a 'src=./website/ dest=/var/www/html/
delete=yes recursive=yes' -b
```

7.5 Command Modules — Choosing the Right One

Ansible provides four modules for running arbitrary commands, and choosing the wrong one is a common beginner mistake. The table below explains the differences.

Module	Shell?	Idempotent?	When to Use
command	No (no shell features)	No (always runs)	Default for simple commands; safe from shell injection
shell	Yes (/bin/sh)	No (always runs)	When you need pipes, redirects, env vars, $HOME
raw	No (no Python on remote)	No	Bootstrapping: install Python on a fresh host
script	Runs a local script remotely	No	Run a complex bash/python script on remote hosts

Table 7.1 — command vs shell vs raw vs script

BASH

```
# command module: no shell features
ansible webservers -m ansible.builtin.command -a 'ls -la /tmp'

# shell module: pipes and redirects work
```

```
ansible webservers -m ansible.builtin.shell -a 'cat /etc/hosts | grep webserver > /tmp/webs'

# raw module: no Python needed on remote (bootstrap)
ansible webservers -m ansible.builtin.raw -a 'yum install -y python3'

# script module: copy and run a local script remotely
ansible webservers -m ansible.builtin.script -a './scripts/install_app.sh'
```

SECURITY · Why command is preferred over shell

The command module does not invoke a shell, which means it is immune to shell injection attacks. If a variable you pass to the command contains ;rm -rf /, the command module will treat that as a literal argument, not as a shell command. Use shell only when you genuinely need shell features (pipes, redirects, globbing, variables) and you have full control over the input.

7.6 Network and Firewall Modules

firewalld — Manage firewalld zones and rules (RHEL family)

BASH

```
# Open ports 80 and 443 in the public zone permanently
ansible webservers -m ansible.posix.firewalld -a 'port=80/tcp permanent=yes state=enabled
immediate=yes' -b
ansible webservers -m ansible.posix.firewalld -a 'port=443/tcp permanent=yes state=enabled
immediate=yes' -b

# Add a service (uses service name instead of port)
ansible webservers -m ansible.posix.firewalld -a 'service=http permanent=yes state=enabled
immediate=yes' -b
```

ufw — Manage UFW rules (Debian family)

BASH

```
ansible webservers -m community.general.ufw -a 'rule=allow port=80 proto=tcp' -b
```

7.7 Cloud and Source Control Modules

git — Clone or update a git repository on remote hosts

BASH

```
# Clone a repository to /opt/myapp on each webserver
ansible webservers -m ansible.builtin.git -a 'repo=https://github.com/example/myapp.git
dest=/opt/myapp version=main' -b
```

7.8 Quick Module Selection Guide

I want to...	Use this module
Check if Ansible can connect	ansible.builtin.ping

I want to...	Use this module
Run a quick command (no shell)	ansible.builtin.command
Run a command with pipes/redirects	ansible.builtin.shell
Install an RPM package	ansible.builtin.yum or dnf
Install a .deb package	ansible.builtin.apt
Start/stop/restart a service	ansible.builtin.service
Create a user	ansible.builtin.user
Create a group	ansible.builtin.group
Create/remove a directory or file	ansible.builtin.file
Copy a file from control to managed	ansible.builtin.copy
Pull a file from managed to control	ansible.builtin.fetch
Sync a directory tree with rsync	ansible.posix.synchronize
Edit one line in a config file	ansible.builtin.lineinfile
Edit a block of lines in a config file	ansible.builtin.blockinfile
Render a Jinja2 template to a file	ansible.builtin.template
Open a firewall port	ansible.posix.firewalld
Clone a git repo	ansible.builtin.git
Get system facts (OS, IP, memory)	ansible.builtin.setup
Reboot and wait	ansible.builtin.reboot

Table 7.2 — Quick module selection guide

MALAYALAM NOTE · Malayalam Teaching Note

Modules ennu parayunnathu Ansible managed node-ilekk kooduthu execute aakkunna Python code pieces aanu. ansible-doc command vechu documentation kaanam — internet venam illa. Common categories: System (user, group, service), Package (yum, apt, pip), File (file, copy, lineinfile, fetch), Command (command vs shell vs raw vs script), Network (firewalld, ufw), Cloud (git). command module shell-ine kaalum safe aanu — shell injection risk illa. Shell module pipes-redirectsvenam njan mathram use cheyyuka.

CHAPTER SUMMARY

Modules are the building blocks of Ansible automation. Each module performs one task idempotently and returns structured JSON output. Use ansible-doc to learn any module offline. Prefer specific modules (yum, service, user, file) over generic command/shell modules for idempotency and safety. The command module is preferred over shell because it is immune to shell injection; use shell only when you need pipes, redirects, or environment variables.

REVIEW QUESTIONS

1. Which module would you use to install an RPM package on AlmaLinux? Give the syntax.
2. Explain the difference between command and shell modules with an example.
3. Why is the raw module useful for bootstrapping a brand-new server?
4. Write an ad-hoc command to ensure the line SELINUX=disabled exists in /etc/selinux/config.
5. Which module would you use to copy a file from a managed node back to the control node?
6. Which module would you use to add port 443/tcp to the public firewalld zone permanently?
7. How can you view documentation for the user module without internet access?

CHAPTER 8

Playbooks Foundations

LEARNING OBJECTIVES

- *Understand the structure of an Ansible playbook (plays, tasks, modules).*
- *Write a complete playbook from scratch and run it with ansible-playbook.*
- *Use --check mode for dry runs and --diff mode for change previews.*
- *Validate playbook syntax before execution.*
- *Observe idempotency in action by running a playbook twice.*

8.1 What is a Playbook?

A playbook is a YAML file that describes the desired state of one or more managed hosts. Where ad-hoc commands are one-shot operations, playbooks are repeatable, version-controllable, and shareable. A playbook contains one or more *plays*; each play targets a group of hosts and contains a list of *tasks*; each task calls one Ansible module with arguments. This three-level hierarchy — playbook, play, task — is the foundation of every Ansible automation project.

MALAYALAM NOTE · Malayalam Teaching Note

Playbook ennu parayunnathu oru YAML file aanu — adhil nammal servers-nte desired state ezhuthum. Ad-hoc command oru kaariyam cheyyum pinnit marum, aanaal playbook repeatable, version-controllable, shareable aanu. Structure: playbook -> play -> task -> module. Oru playbook-il multiple plays undaakum, oru play-il multiple tasks, oru task oru module call cheyyum.

8.2 Anatomy of a Playbook

Every playbook starts with --- (the YAML document marker) and contains a list of plays. Each play begins with a - name: and includes a hosts: directive that selects the target group, optional play-level directives like become:, and a tasks: section listing the tasks to execute in order.

YAML

```yaml
---
- name: Install and start nginx on webservers
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx package
      ansible.builtin.yum:
        name: nginx
        state: present
```

```
    - name: Ensure nginx is started and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes

    - name: Open port 80 in firewalld
      ansible.posix.firewalld:
        service: http
        permanent: yes
        state: enabled
        immediate: yes
```

Save this as ~/ansible-lab/install_nginx.yml. Notice the indentation: two spaces per level, never tabs. The name: keys make the playbook self-documenting — Ansible will print these names as it runs, so always write descriptive names like *Install nginx package* rather than vague ones like *do thing*.

8.3 Running a Playbook

Run the playbook with the ansible-playbook command. The output shows each task's result with a coloured status: green for OK (no change needed), yellow for changed, red for failure. At the end you get a summary like PLAY RECAP showing how many hosts succeeded, failed, or changed.

BASH

```
cd ~/ansible-lab
ansible-playbook install_nginx.yml
```

EXPECTED OUTPUT

```
PLAY [Install and start nginx on webservers] *********************************

TASK [Install nginx package] *************************************************
changed: [webserver1]
changed: [webserver2]

TASK [Ensure nginx is started and enabled] ***********************************
changed: [webserver1]
changed: [webserver2]

TASK [Open port 80 in firewalld] *********************************************
ok: [webserver1]
ok: [webserver2]

PLAY RECAP *******************************************************************
webserver1                 :
ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
webserver2                 :
ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

8.4 Idempotency in Action — Run It Twice

Now run the exact same playbook a second time. Because every task is idempotent, Ansible should report ok=3 changed=0 — nothing changed because nginx is already installed, running, and the firewall port is already open. This is the magic of idempotent modules: you can run the playbook a hundred times and only the first run actually changes anything.

BASH

```
ansible-playbook install_nginx.yml
```

EXPECTED OUTPUT

```
PLAY RECAP *********************************************************************
webserver1                 : ok=3    changed=0    unreachable=0    failed=0
webserver2                 : ok=3    changed=0    unreachable=0    failed=0
```

> **TIP · Idempotency is your safety net**
>
> If you ever see changed=2 on the second run of an idempotent playbook, something is wrong. Either a task is not idempotent (you used the shell module without changed_when), or another process is reverting the changes between runs. Investigate before running in production.

8.5 Check Mode (Dry Run)

Before running a playbook against production, always test it with --check. Check mode simulates every task without actually making changes. Ansible still connects to the hosts, still gathers facts, still evaluates conditionals, but the modules report what they *would* change rather than actually changing it. This is invaluable for safe deployment.

BASH

```
ansible-playbook install_nginx.yml --check
```

8.6 Diff Mode

Add --diff to see line-by-line diffs of any file or template changes the playbook would make. Combined with --check, this gives you a complete preview of the changes before they happen — the closest thing to a git diff for infrastructure.

BASH

```
ansible-playbook install_nginx.yml --check --diff
```

8.7 Syntax Validation

Always validate playbook syntax before running. YAML is whitespace-sensitive, and a single wrong indent can cause cryptic errors. The --syntax-check flag parses the playbook without executing it — the fastest way to catch typos.

BASH

```
ansible-playbook install_nginx.yml --syntax-check
```

EXPECTED OUTPUT

```
playbook: install_nginx.yml
```

For deeper linting, install ansible-lint (a separate Python package) and run it against your playbooks. It catches best-practice violations, deprecated syntax, and other issues that syntax-check misses.

BASH

```
pip install ansible-lint
ansible-lint install_nginx.yml
```

8.8 Common Playbook Mistakes

Mistake	Symptom	Fix
Tabs instead of spaces	YAML parse error	Use 2 spaces per indent level; configure editor to use spaces
Missing colon after key	YAML parse error or unexpected behaviour	Always write key: value with the space after colon
Unquoted asterisks or colons in values	YAML interprets as alias or mapping	Quote values containing special characters
Forgetting become: for privileged tasks	Permission denied errors	Set become: true at play level or in ansible.cfg
Task name omitted	Output shows module name instead of description	Always add a descriptive name: to every task
Wrong module name	Module not found error	Use full FQCN like ansible.builtin.yum

Table 8.1 — Common playbook mistakes

CHAPTER SUMMARY

A playbook is a YAML file containing one or more plays, each targeting a host group and listing tasks that call modules. Run playbooks with ansible-playbook. Idempotent modules report changed=0 on the second run — this is the key safety guarantee. Always validate syntax with --syntax-check, preview changes with --check --diff, and lint with ansible-lint before running in production.

REVIEW QUESTIONS

1. Describe the three-level hierarchy of an Ansible playbook.
2. What is the difference between green, yellow, and red status lines in playbook output?
3. Why does running the same playbook twice result in changed=0 on the second run?
4. Which flag performs a dry run? Which flag shows file diffs?
5. Why is --syntax-check valuable even before --check?
6. Write a one-play playbook that installs the git package on the webservers group with sudo.

CHAPTER 9

Variables, Facts, and Magic Variables

LEARNING OBJECTIVES

- *Define variables at multiple levels: inventory, group_vars, host_vars, playbook vars, extra vars.*
- *Understand variable precedence (22 levels — simplified list).*
- *Use register to capture task output and reuse it later.*
- *Use magic variables: inventory_hostname, groups, group_names, hostvars, ansible_play_hosts.*
- *Use facts (ansible_distribution, ansible_default_ipv4, ansible_memtotal_mb) in playbooks.*
- *Disable fact gathering with gather_facts: no for speed.*

9.1 Why Variables?

Hard-coding values in playbooks is brittle. If you write name: nginx in ten different tasks, changing the package name later means editing ten lines. Variables solve this by letting you define a value once and reference it everywhere. Ansible variables can hold strings, numbers, lists, dictionaries, and even nested structures. They are referenced in templates and task arguments with {{ variable_name }} (Jinja2 syntax).

9.2 Defining Variables — Where and How

Variables can be defined in many places, each with a different precedence. The most common are listed below in increasing order of precedence (later wins over earlier).

- **Role defaults** — in roles//defaults/main.yml. Lowest precedence; sensible defaults.
- **Inventory file vars** — in [group:vars] or per-host.
- **group_vars/ and host_vars/** — playbook-adjacent directories.
- **Playbook vars** — in the play: vars: { key: value }.
- **vars_files** — included YAML files: vars_files: [secrets.yml].
- **vars_prompt** — interactive prompts at run time.
- **Extra vars (--extra-vars or -e)** — CLI overrides. Highest precedence.

9.3 Variable Precedence Table (Simplified)

Ansible's full precedence list has 22 levels, but in practice you only need to remember the simplified table below. Higher number wins over lower.

Priority	Source	Typical Use
1 (lowest)	role defaults	Default values that roles provide
2	inventory group_vars/all	Site-wide defaults
3	inventory group_vars/	Group-specific values
4	inventory host_vars/	Host-specific values
5	playbook vars	Play-level variables
6	vars_files	External variable files (often vault-encrypted)
7	vars_prompt	Interactive prompts at run time
8 (highest)	extra vars (--extra-vars / -e)	CLI overrides for one-off runs

Table 9.1 — Variable precedence (simplified)

9.4 Defining Variables in a Playbook

YAML

```
---
- name: Demonstrate variables in a playbook
  hosts: webservers
  become: true
  vars:
    nginx_port: 8080
    doc_root: /var/www/html
    welcome_message: "Hello from {{ inventory_hostname }}"
  tasks:
    - name: Create document root
      ansible.builtin.file:
        path: "{{ doc_root }}"
        state: directory
        mode: '0755'

    - name: Deploy index.html with the welcome message
      ansible.builtin.copy:
        content: "{{ welcome_message }}\n"
        dest: "{{ doc_root }}/index.html"
        mode: '0644'
```

Notice that values containing variable references must be quoted ("{{ doc_root }}/index.html") because the curly braces are also YAML flow-mapping syntax. Always quote when in doubt.

9.5 Loading Variables from Files (vars_files)

For larger playbooks, inline vars become unwieldy. Use vars_files to load variables from external YAML files. This also lets you encrypt sensitive variables with Ansible Vault (Chapter 14) while keeping the playbook itself unencrypted.

YAML

```
---
- name: Load variables from external files
```

```
  hosts: webservers
  become: true
  vars_files:
    - vars/common.yml
    - vars/webservers.yml
    - vars/secrets.vault.yml     # vault-encrypted
  tasks:
    - name: Show a variable loaded from file
      ansible.builtin.debug:
        var: db_password
```

9.6 Interactive Prompts (vars_prompt)

For sensitive values that should not be stored in files (like a one-time API key), use vars_prompt to ask the operator at run time.

YAML

```yaml
---
- name: Ask for a value at run time
  hosts: webservers
  vars_prompt:
    - name: admin_password
      prompt: Enter the admin password
      private: yes        # do not echo to terminal
      confirm: yes        # ask twice
  tasks:
    - name: Use the prompted password
      ansible.builtin.user:
        name: admin
        password: "{{ admin_password | password_hash('sha512') }}"
```

9.7 Register — Capturing Task Output

The register keyword captures the result of a task into a variable you can use later. This is essential for any playbook that needs to make decisions based on the output of a previous command.

YAML

```yaml
---
- name: Capture command output
  hosts: webservers
  tasks:
    - name: Check if /etc/motd exists
      ansible.builtin.stat:
        path: /etc/motd
      register: motd_stat

    - name: Show whether motd exists
      ansible.builtin.debug:
        msg: "motd exists: {{ motd_stat.stat.exists }}"

    - name: Run a shell command and capture output
      ansible.builtin.command: uname -r
      register: kernel_version
```

```
    - name: Print the kernel version
      ansible.builtin.debug:
        var: kernel_version.stdout
```

Registered variables have rich structure — they include stdout, stderr, rc (return code), changed, cmd, and more. Always check the documentation for the specific module to know which fields it returns.

9.8 Magic Variables

Magic variables are special variables that Ansible makes available in every playbook without you defining them. They provide information about the inventory and the current run.

Magic Variable	Description	Example Value
inventory_hostname	Name of the current host as listed in inventory	webserver1
groups	Dictionary of all groups and their hosts	{'webservers': ['webserver1', 'webserver2'], ...}
group_names	List of groups the current host belongs to	['webservers', 'datacentre']
hostvars	Dictionary of all hosts and their variables	hostvars["webserver1"]["ansible_host"]
ansible_play_hosts	List of hosts in the current play	['webserver1', 'webserver2']
ansible_play_batch	List of hosts in the current batch (fork-limited)	['webserver1']
play_hosts	Alias for ansible_play_hosts (older)	Same as above
omit	Special variable to skip a parameter	mode: "{{ mode | default(omit) }}"

Table 9.2 — Common Ansible magic variables

YAML

```
---
- name: Show magic variables
  hosts: webservers
  tasks:
    - name: Print current host name
      ansible.builtin.debug:
        msg: "I am running on {{ inventory_hostname }}"

    - name: Print all hosts in the webservers group
      ansible.builtin.debug:
        var: groups['webservers']

    - name: Print webserver1 IP from hostvars
      ansible.builtin.debug:
        var: hostvars['webserver1']['ansible_host']
```

9.9 Facts — System Information from setup Module

Facts are variables that Ansible automatically discovers about each managed host at the start of a play (via the setup module). They include the operating system, IP addresses, memory, disks, CPU, mounted filesystems, and much more. Facts are stored as variables prefixed with ansible_ and are available in tasks and templates.

BASH

```
# See all facts for a host
ansible webserver1 -m ansible.builtin.setup

# Filter for specific facts
ansible webserver1 -m ansible.builtin.setup -a 'filter=ansible_distribution*'
ansible webserver1 -m ansible.builtin.setup -a 'filter=ansible_memtotal_mb'
```

Fact	Description	Example Value
ansible_distribution	OS name	AlmaLinux
ansible_distribution_version	OS version	9.3
ansible_os_family	OS family (RedHat, Debian, Suse, etc.)	RedHat
ansible_hostname	Short hostname	webserver1
ansible_fqdn	Fully qualified hostname	webserver1.example.com
ansible_default_ipv4.address	Default IPv4 address	192.168.1.21
ansible_memtotal_mb	Total RAM in MB	2048
ansible_processor_vcpus	Number of CPU cores	2
ansible_architecture	CPU architecture	x86_64
ansible_kernel	Kernel version	5.14.0-362.el9.x86_64

Table 9.3 — Commonly used Ansible facts

9.10 Using Facts in a Playbook

YAML

```
---
- name: Use facts to install the right package
  hosts: webservers
  become: true
  tasks:
    - name: Install Apache on RedHat family
      ansible.builtin.yum:
        name: httpd
        state: present
      when: ansible_os_family == 'RedHat'

    - name: Install Apache on Debian family
      ansible.builtin.apt:
        name: apache2
        state: present
```

```
      when: ansible_os_family == 'Debian'

    - name: Print system info
      ansible.builtin.debug:
        msg: "{{ ansible_distribution }} {{ ansible_distribution_version }} on {{
ansible_memtotal_mb }}MB RAM"
```

9.11 Disabling Fact Gathering for Speed

Fact gathering takes 1-3 seconds per host. For playbooks that do not use any facts, you can disable it with gather_facts: no at the play level. This is especially useful when running a playbook against hundreds of hosts.

YAML

```
---
- name: Fast playbook without fact gathering
  hosts: webservers
  gather_facts: no
  tasks:
    - name: Quick ping
      ansible.builtin.ping:
```

9.12 Custom Facts (facts.d)

You can extend Ansible's facts with your own by placing executable scripts or static JSON files in /etc/ansible/facts.d/ on the managed host. Ansible executes any file ending in .fact and merges the JSON output into ansible_local. This is a powerful way to expose application-specific information to Ansible.

BASH

```
# On managed node: /etc/ansible/facts.d/app_version.fact
#!/bin/bash
echo '{"version": "2.4.1", "deployed": "2024-10-24"}'

chmod +x /etc/ansible/facts.d/app_version.fact

# On control node: reference the custom fact
- name: Show custom fact
  ansible.builtin.debug:
    var: ansible_local.app_version.version
```

MALAYALAM NOTE · Malayalam Teaching Note

Variables nammale avoid cheyyum hard-coding. Different levels-il variables define cheyyam — inventory, group_vars, host_vars, playbook vars, extra vars. Precedence: extra vars highest, role defaults lowest. register keyword vechu task output capture cheyyam. Magic variables (inventory_hostname, groups, hostvars) automatic aayi kittum. Facts setup module ninnu kittum — OS, IP, memory etc. gather_facts: no vecha speed-up cheyyam.

CHAPTER SUMMARY

Variables make playbooks reusable by avoiding hard-coded values. They can be defined in 22 precedence levels (extra vars highest, role defaults lowest). The register keyword captures task output for later use. Magic variables (inventory_hostname, groups, hostvars) provide inventory context. Facts are auto-discovered system information (OS, IP, memory) accessible as ansible_* variables. Disable gather_facts: no when facts are not needed for speed. Custom facts can be added via /etc/ansible/facts.d/ on managed hosts.

REVIEW QUESTIONS

1 List five places where Ansible variables can be defined, in increasing order of precedence.

2 What is the difference between vars and vars_files in a play?

3 Write a task that registers the output of df -h and then prints the stdout.

4 Which magic variable contains the name of the current host as listed in the inventory?

5 Which fact would you check to determine whether a host is RedHat-family or Debian-family?

6 Why might you set gather_facts: no on a playbook?

7 How can you add custom facts that are specific to your application?

CHAPTER 10

Conditionals, Loops, and Task Control

LEARNING OBJECTIVES

- *Use when conditionals to skip or run tasks based on facts, variables, or registered output.*
- *Use loops (loop, with_items, with_dict, with_fileglob) to repeat tasks.*
- *Use changed_when, failed_when, and ignore_errors to control task status.*
- *Use tags to run subsets of a playbook.*
- *Use block/rescue/always for try-except-finally error handling.*

10.1 The when Conditional

The when keyword conditionally executes a task. The condition is a Jinja2 expression that evaluates to true or false. If true, the task runs; if false, the task is skipped. Conditions can compare variables, check facts, test registered output, or combine multiple expressions with and, or, and not.

YAML

```yaml
---
- name: Conditional task examples
  hosts: webservers
  become: true
  tasks:
    - name: Install httpd only on RedHat family
      ansible.builtin.yum:
        name: httpd
        state: present
      when: ansible_os_family == 'RedHat'

    - name: Install apache2 only on Debian family
      ansible.builtin.apt:
        name: apache2
        state: present
      when: ansible_os_family == 'Debian'

    - name: Run only on primary webserver
      ansible.builtin.debug:
        msg: "I am the primary server"
      when: inventory_hostname == 'webserver1'

    - name: Run if memory is more than 4GB
      ansible.builtin.debug:
        msg: "Plenty of memory: {{ ansible_memtotal_mb }}MB"
      when: ansible_memtotal_mb > 4096
```

10.2 Combining Conditions

Use and, or, and not to combine conditions. Parentheses are not required but improve readability.

YAML

```
    - name: Run on AlmaLinux OR Rocky with more than 2GB RAM
      ansible.builtin.debug:
        msg: "Compatible host"
      when:
        (ansible_distribution == 'AlmaLinux' or ansible_distribution == 'Rocky')
        and ansible_memtotal_mb > 2048

    - name: Run only if the variable is defined and non-empty
      ansible.builtin.debug:
        var: my_var
      when: my_var is defined and my_var | length > 0
```

10.3 when with Registered Variables

Useful patterns include checking whether a registered command succeeded, failed, or produced specific output.

YAML

```
    - name: Check if app is installed
      ansible.builtin.command: which myapp
      register: app_check
      ignore_errors: true    # do not fail the play on non-zero exit

    - name: Install app if not present
      ansible.builtin.yum:
        name: myapp
        state: present
      when: app_check.rc != 0   # rc is the return code

    - name: Skip when previous task succeeded
      ansible.builtin.debug:
        msg: "App was already installed, skipping"
      when: app_check.rc == 0
```

10.4 Loops — The loop Keyword

The loop keyword (and its older cousins with_items, with_dict, etc.) repeats a task once per item in a list. Each item is available as {{ item }} inside the task.

YAML

```
---
- name: Loop examples
  hosts: webservers
  become: true
  tasks:
    - name: Install multiple packages
      ansible.builtin.yum:
        name: "{{ item }}"
        state: present
      loop:
        - nginx
```

```
      - git
      - vim
      - curl

  - name: Create multiple users
    ansible.builtin.user:
      name: "{{ item.name }}"
      groups: "{{ item.groups }}"
      shell: /bin/bash
    loop:
      - { name: 'alice', groups: 'wheel' }
      - { name: 'bob',   groups: 'developers' }
      - { name: 'carol', groups: 'developers' }
```

10.5 Looping Over Dictionaries

YAML

```
  - name: Create directories from a dict
    ansible.builtin.file:
      path: "{{ item.value.path }}"
      owner: "{{ item.value.owner }}"
      state: directory
      mode: "{{ item.value.mode }}"
    loop: "{{ site_directories | dict2items }}"
    vars:
      site_directories:
        html:
          path: /var/www/html
          owner: nginx
          mode: '0755'
        logs:
          path: /var/log/myapp
          owner: nginx
          mode: '0750'
```

10.6 Looping Over Files (with_fileglob)

YAML

```
  - name: Copy all .conf files from local directory
    ansible.builtin.copy:
      src: "{{ item }}"
      dest: /etc/nginx/conf.d/
      owner: root
      group: root
      mode: '0644'
    with_fileglob:
      - files/nginx/*.conf
```

10.7 until — Retry Until a Condition is Met

The until keyword retries a task until a condition is met or the retries limit is hit. Useful for waiting for a service to come up.

YAML

```yaml
    - name: Wait for web server to respond
      ansible.builtin.uri:
        url: http://localhost:8080/health
        return_content: yes
      register: health_check
      until: health_check.status == 200
      retries: 10
      delay: 5    # seconds between retries
```

10.8 Tags — Running Subsets of a Playbook

Tags let you mark tasks so you can run only specific subsets of a playbook. This is extremely useful for large playbooks where you only want to re-run the database configuration, or only the web server restart.

YAML

```yaml
---
- name: Playbook with tags
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.yum:
        name: nginx
        state: present
      tags: install

    - name: Configure nginx
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      tags: configure

    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
      tags: restart
```

BASH

```bash
# Run only tasks tagged 'configure'
ansible-playbook site.yml --tags configure

# Run everything except 'restart'
ansible-playbook site.yml --skip-tags restart

# List all tags in a playbook without running it
ansible-playbook site.yml --list-tags
```

10.9 block/rescue/always — Error Handling

The block keyword groups tasks together and lets you handle errors gracefully. Tasks inside block run normally. If any of them fails, tasks inside rescue run instead. Tasks inside always run regardless of outcome. This is Ansible's equivalent of try/except/finally.

YAML

```yaml
- name: Try to deploy, roll back on failure
  block:
    - name: Deploy new app version
      ansible.builtin.copy:
        src: app-v2.tar.gz
        dest: /opt/app/app.tar.gz

    - name: Restart app service
      ansible.builtin.service:
        name: app
        state: restarted

  rescue:
    - name: Roll back to previous version
      ansible.builtin.copy:
        src: app-v1.tar.gz
        dest: /opt/app/app.tar.gz
        remote_src: no

    - name: Restart app service (rolled back)
      ansible.builtin.service:
        name: app
        state: restarted

    - name: Notify the team about the rollback
      ansible.builtin.debug:
        msg: "Deployment failed and was rolled back on {{ inventory_hostname }}"

  always:
    - name: Clean up temporary files
      ansible.builtin.file:
        path: /tmp/deploy_temp
        state: absent
```

10.10 changed_when, failed_when, ignore_errors

Sometimes a module reports changed when nothing actually changed, or fails when the failure is expected. These keywords let you override the default behaviour.

YAML

```yaml
- name: Run a command that always reports 'changed'
  ansible.builtin.command: date
  changed_when: false    # date does not change system state

- name: Run a command that may fail and that's OK
  ansible.builtin.command: pgrep nginx
  ignore_errors: true
  register: nginx_proc

- name: Custom failure condition
  ansible.builtin.command: /usr/local/bin/health-check
  register: health
  failed_when: "'ERROR' in health.stdout"

- name: Custom change condition
  ansible.builtin.command: /usr/local/bin/sync-config
```

```
    register: sync
    changed_when: "'changed' in sync.stdout"
```

MALAYALAM NOTE · Malayalam Teaching Note

Conditionals (when) vechu task skip cheyyam — OS family, memory, variables check cheyyam. Loops (loop, with_items) vechu same task multiple items-ekku repeat cheyyam — item.name, item.value etc. Tags vechu playbook-le subsets run cheyyam — --tags, --skip-tags. block/rescue/always venchu try-except-finally pattern — rescue vachu rollback cheyyam. changed_when, failed_when, ignore_errors vachu task status control cheyyam.

CHAPTER SUMMARY

The when keyword conditionally runs tasks based on facts, variables, or registered output. Loops (loop, with_items, with_dict, with_fileglob) repeat tasks over collections. Tags allow running subsets of a playbook. The block/rescue/always construct provides try-except-finally error handling for graceful rollbacks. changed_when, failed_when, and ignore_errors give fine-grained control over task status reporting.

REVIEW QUESTIONS

1. Write a task that installs httpd only when the OS family is RedHat.
2. Write a task that creates three users — alice, bob, and carol — using a loop.
3. Explain the difference between ignore_errors: true and rescue.
4. How would you wait for a URL to return HTTP 200, retrying up to 10 times with 5-second delays?
5. Write a block/rescue/always that attempts to copy a file, rolls back on failure, and always cleans up.
6. Which flag runs only tasks tagged configure?
7. Why would you use changed_when: false on a command that just prints the date?

CHAPTER 11

Templates with Jinja2

LEARNING OBJECTIVES

- *Understand Jinja2 syntax: {{ var }}, {% if %}, {% for %}, {# comment #}.*
- *Use common filters: default, upper, lower, replace, regex_replace, to_json, json_query.*
- *Create a Jinja2 template file (.j2) and render it via the template module.*
- *Use conditionals and loops inside templates.*
- *Validate template syntax.*

11.1 What is Jinja2?

Jinja2 is a templating engine for Python. Ansible uses it for two purposes: rendering templates (files with placeholders that get filled with variable values) and evaluating expressions in playbooks (the {{ ... }} syntax you have already seen). Jinja2 supports variables, conditionals, loops, filters, and macros — enough to generate any configuration file dynamically.

11.2 Jinja2 Syntax Basics

Syntax	Purpose	Example
{{ expression }}	Print the value of an expression	{{ nginx_port }}
{% statement %}	Control flow (if, for, set)	{% if ansible_os_family == "RedHat" %}
{# comment #}	Comment (not in output)	{# This is a comment #}
{{ var | filter }}	Apply a filter to a variable	{{ name | upper }}
{{ var | default("x") }}	Default value if var is undefined	{{ port | default(80) }}

Table 11.1 — Jinja2 syntax basics

11.3 Common Filters

Filter	Purpose	Example
default(x)	Provide a default if undefined	{{ port | default(80) }}
upper / lower	Convert case	{{ name | upper }}

Filter	Purpose	Example
replace(a, b)	String substitution	{{ msg | replace("old", "new") }}
regex_replace(pat, repl)	Regex substitution	{{ ip | regex_replace("^([0-9]+)\..*", "\1") }}
trim	Strip whitespace	{{ name | trim }}
length	Length of string or list	{{ users | length }}
first / last	First/last element of list	{{ hosts | first }}
sort / reverse	Sort or reverse a list	{{ ports | sort }}
unique	Remove duplicates	{{ tags | unique }}
join(sep)	Join list into string	{{ hosts | join(",") }}
to_json / to_nice_json	Convert to JSON	{{ config | to_nice_json }}
from_json	Parse JSON string	{{ '{"a":1}' | from_json }}
list	Convert iterable to list	{{ range(5) | list }}
dict2items	Convert dict to list of {key,value}	{{ mydict | dict2items }}
items2dict	Reverse of dict2items	{{ items | items2dict }}
password_hash(algo)	Hash a password	{{ 'secret' | password_hash('sha512') }}

Table 11.2 — Common Jinja2 filters

11.4 Creating a Template File

A template file is a regular text file with Jinja2 placeholders, saved with the .j2 extension by convention. Let us create an nginx configuration template that customises the listening port, document root, and server name per host.

NGINX

```
# ~/ansible-lab/templates/nginx.conf.j2
# Managed by Ansible - do not edit manually
# Generated for {{ inventory_hostname }} ({{ ansible_default_ipv4.address }})

worker_processes auto;

events {
    worker_connections {{ worker_connections | default(1024) }};
}

http {
    server {
        listen {{ nginx_port }};
        server_name {{ ansible_hostname }}.example.com;
        root {{ doc_root }};
        index index.html index.htm;
```

```
        location / {
            try_files $uri $uri/ =404;
        }

        # Per-host configuration
        location /status {
            stub_status on;
            {% if enable_status_auth | default(false) %}
            auth_basic "Status";
            auth_basic_user_file /etc/nginx/.htpasswd;
            {% endif %}
        }
    }
}
```

11.5 Rendering the Template with the template Module

The template module reads a .j2 file from the control node, renders it with the current variables and facts, and writes the result to the managed host. It is idempotent: if the rendered content matches the existing file, Ansible reports ok and does not rewrite.

YAML

```yaml
---
- name: Deploy nginx config from template
  hosts: webservers
  become: true
  vars:
    nginx_port: 8080
    doc_root: /var/www/html
    worker_connections: 2048
  tasks:
    - name: Render nginx.conf
      ansible.builtin.template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
        backup: yes
      notify: restart nginx

  handlers:
    - name: restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

11.6 Conditionals Inside Templates

NGINX

```
# Example: conditionally include upstream servers
upstream app_servers {
    {% for host in groups['webservers'] %}
    server {{ hostvars[host]['ansible_host'] }}:8000;
```

```
    {% endfor %}
}

# Example: if-elif-else
{% if ansible_os_family == 'RedHat' %}
    include /etc/nginx/conf.d/*.conf;
{% elif ansible_os_family == 'Debian' %}
    include /etc/nginx/sites-enabled/*;
{% else %}
    # Unknown OS family
{% endif %}
```

11.7 Loops Inside Templates

NGINX

```
# Generate a vhost file per virtual host
{% for vhost in virtual_hosts %}
server {
    listen 80;
    server_name {{ vhost.name }};
    root {{ vhost.root }};

    location / {
        try_files $uri $uri/ =404;
    }
}
{% endfor %}
```

11.8 Generating /etc/motd with System Info

A common use of templates is to generate a custom login banner showing system information. This makes it immediately obvious which server you have SSHed into.

TEXT

```
# ~/ansible-lab/templates/motd.j2
########################################
#  Welcome to {{ ansible_hostname }}
#
#  OS:       {{ ansible_distribution }} {{ ansible_distribution_version }}
#  Kernel:   {{ ansible_kernel }}
#  CPU:      {{ ansible_processor_vcpus }} vCPUs
#  Memory:   {{ ansible_memtotal_mb }} MB
#  IP:       {{ ansible_default_ipv4.address }}
#
#  Managed by Ansible - do not edit manually
########################################
```

YAML

```
---
- name: Deploy custom motd
  hosts: webservers
  become: true
  tasks:
    - name: Render motd
```

```
    ansible.builtin.template:
      src: templates/motd.j2
      dest: /etc/motd
      owner: root
      group: root
      mode: '0644'
```

11.9 Validating Template Syntax

Use --check with the playbook to validate the template would render without errors. For deeper inspection, use ansible -m debug -a 'msg={{ lookup("template", "templates/nginx.conf.j2") }}' to see the rendered output without writing it to disk.

BASH

```
ansible-playbook deploy_nginx.yml --check --diff
```

MALAYALAM NOTE · Malayalam Teaching Note

Jinja2 templates vachu nammal configuration files dynamically generate cheyyum. Syntax: {{ var }} for printing, {% if %} {% for %} for control flow, {# comment #} for comments. Filters use cheyyam — default, upper, replace, regex_replace, to_json, password_hash. Template files .j2 extension vechu save cheyyum. template module vachu render cheyyum — idempotent aanu. Conditionals loops templates-il use cheyyam.

CHAPTER SUMMARY

Jinja2 is the templating engine Ansible uses for rendering configuration files. Templates are saved as .j2 files containing placeholders and control flow. The template module renders them on the control node and deploys the result. Common filters include default, upper, replace, regex_replace, to_json, and password_hash. Always validate templates with --check --diff before deploying to production.

REVIEW QUESTIONS

1. What is the difference between {{ var }}, {% if %}, and {# comment #}?
2. Write a Jinja2 expression that prints the value of port or 80 if undefined.
3. Write a Jinja2 loop that generates one server line per host in the webservers group.
4. Which Ansible module renders a .j2 template to a remote file?
5. How would you check what a template will produce before deploying it?
6. Which filter would you use to hash a password for the /etc/shadow file?

CHAPTER 12

Handlers and Error Handling

LEARNING OBJECTIVES

- *Understand what handlers are and when to use them.*
- *Use the notify keyword to trigger handlers.*
- *Force handler execution with meta: flush_handlers and --force-handlers.*
- *Use block/rescue/always for graceful error handling.*
- *Use ignore_errors, failed_when, changed_when, any_errors_fatal.*

12.1 What is a Handler?

A handler is a special kind of task that runs only when notified by another task. Handlers are typically used to restart a service after its configuration file changes. The key benefit is idempotency: if the config file is unchanged (task reports ok), the handler does not run, so the service is not needlessly restarted. If the config file is changed (task reports changed), the handler runs once at the end of the play.

12.2 Defining and Notifying Handlers

YAML

```yaml
---
- name: Configure nginx with handler
  hosts: webservers
  become: true
  tasks:
    - name: Deploy nginx config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: restart nginx     # handler name

    - name: Deploy site config
      ansible.builtin.template:
        src: site.conf.j2
        dest: /etc/nginx/conf.d/site.conf
      notify: restart nginx     # same handler

    - name: Open firewall port
      ansible.posix.firewalld:
        service: http
        permanent: yes
        state: enabled
        immediate: yes
      notify: reload firewalld
```

```
handlers:
  - name: restart nginx
    ansible.builtin.service:
      name: nginx
      state: restarted

  - name: reload firewalld
    ansible.builtin.systemd:
      name: firewalld
      state: reloaded
```

Note several key points. First, both config-deploy tasks notify the same handler (restart nginx). The handler runs **only once**, even if notified twice — Ansible deduplicates. Second, the handler runs at the end of the play by default, after all tasks have completed. Third, if neither task changes anything, the handler does not run at all — this is the idempotency benefit.

12.3 Handler Ordering — A Common Pitfall

Handlers run in the order they are **defined** in the handlers section, not in the order they are notified. This is a frequent source of confusion. If you have notify: [restart nginx, restart php-fpm], but in the handlers section restart php-fpm is defined first, php-fpm will restart first. Always define handlers in the order you want them to run.

12.4 Forcing Handler Execution Early

Sometimes you need a handler to run mid-play rather than at the end. Use meta: flush_handlers as a task to force all pending handlers to run immediately at that point in the playbook.

YAML

```
  - name: Deploy nginx config
    ansible.builtin.template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: restart nginx

  - name: Force handler to run now
    ansible.meta.flush_handlers:

  - name: Curl the local nginx to verify (only works if restart already happened)
    ansible.builtin.uri:
      url: http://localhost/
      return_content: yes
    register: result
```

12.5 Forcing Handlers on Failure

By default, if a task fails, pending handlers do not run. This can leave a service in an inconsistent state if its config was partially updated. Use --force-handlers on the command line to force handlers to run even on failure. This is useful for cleanup handlers.

BASH

```
ansible-playbook deploy.yml --force-handlers
```

12.6 Error Handling with block/rescue/always

We saw block/rescue/always in Chapter 10. Here we expand on the error-handling use case. block contains the main tasks; rescue runs only if any task in block fails; always runs regardless of outcome. Inside rescue, the special variable ansible_failed_result contains the failure details of the task that triggered the rescue.

YAML

```
    - name: Attempt a risky deployment
      block:
        - name: Stop the service
          ansible.builtin.service:
            name: myapp
            state: stopped

        - name: Deploy new code
          ansible.builtin.unarchive:
            src: myapp-v2.tar.gz
            dest: /opt/myapp/

        - name: Start the service
          ansible.builtin.service:
            name: myapp
            state: started

      rescue:
        - name: Restore previous version
          ansible.builtin.copy:
            src: /opt/myapp/backup/myapp-v1/
            dest: /opt/myapp/
            remote_src: yes

        - name: Start the service (rolled back)
          ansible.builtin.service:
            name: myapp
            state: started

        - name: Send alert
          ansible.builtin.debug:
            msg: "Deployment failed on {{ inventory_hostname }}: {{ ansible_failed_result.msg
}}"

      always:
        - name: Remove temporary files
          ansible.builtin.file:
            path: /tmp/myapp_deploy
            state: absent
```

12.7 Controlling Failure Behaviour

Directive	Purpose	Example
ignore_errors: true	Continue play even if task fails	Common for non-critical commands
failed_when: cond	Mark as failed based on custom condition	failed_when: "ERROR" in result.stdout
changed_when: cond	Mark as changed based on custom condition	changed_when: false (never report changed)
any_errors_fatal: true	All hosts fail if any host fails	Critical operations where partial state is bad
max_fail_percentage: N	Fail if more than N% of hosts fail	max_fail_percentage: 25
--force-handlers	Run handlers even on failure	CLI flag for cleanup guarantees

Table 12.1 — Failure behaviour control directives

12.8 Debugging with the debug Module

The debug module is your primary tool for inspecting variables and task output during playbook development. It can print a message, a variable, or both.

YAML

```yaml
    - name: Print a variable
      ansible.builtin.debug:
        var: ansible_distribution

    - name: Print a custom message
      ansible.builtin.debug:
        msg: "Running on {{ inventory_hostname }} ({{ ansible_default_ipv4.address }})"

    - name: Print registered variable
      ansible.builtin.debug:
        var: result.stdout_lines

    - name: Print with verbosity (only shows with -v or higher)
      ansible.builtin.debug:
        var: result
        verbosity: 1
```

MALAYALAM NOTE · Malayalam Teaching Note

Handlers ennu parayunnathu special tasks aanu — only when notified. Common use: service restart cheyyan config change aakumbol. notify keyword vachu trigger cheyyum. Handlers end-of-play-l run aakkum, deduplicate aakkum (twice notify cheyynjal once matram run). Handler order defined order aayirikkum, notify order alla. meta: flush_handlers vachu mid-play run cheyyam. block/rescue/always vachu error handle cheyyam — rescue ansible_failed_result variable access cheyyam.

CHAPTER SUMMARY

Handlers are special tasks that run only when notified by another task, typically to restart a service after config changes. They run once at end of play, in defined order, only if at least one notifying task reports changed. Use meta: flush_handlers to force mid-play execution, and --force-handlers to run on failure. For broader error handling, use block/rescue/always (try/except/finally). Control failure behaviour with ignore_errors, failed_when, changed_when, any_errors_fatal, and max_fail_percentage. Use the debug module to inspect variables during development.

REVIEW QUESTIONS

1 Why are handlers preferred over restarting a service in a regular task?

2 If two tasks notify the same handler, how many times does the handler run?

3 In what order do handlers run — the order they are notified, or the order they are defined?

4 Which task forces pending handlers to run mid-play?

5 Write a block/rescue/always that attempts to start a service, prints an error on failure, and always cleans up a temp file.

6 What does any_errors_fatal: true do? When would you use it?

7 Which module is your primary tool for inspecting variables during development?

CHAPTER 13

Ansible Roles and Galaxy

LEARNING OBJECTIVES

- *Understand the role directory layout: tasks, handlers, defaults, vars, files, templates, meta.*
- *Create a role with ansible-galaxy init.*
- *Use roles in playbooks via roles:, include_role, and import_role.*
- *Define role dependencies in meta/main.yml.*
- *Search and install roles from Ansible Galaxy.*
- *Use requirements.yml to manage role dependencies.*

13.1 Why Roles?

As your playbooks grow, you will find yourself repeating the same sets of tasks in multiple playbooks — install nginx, configure it, start it. Copying and pasting is brittle; if you fix a bug in one copy, you must remember to fix it everywhere. **Roles** solve this by letting you package related tasks, handlers, variables, files, and templates into a reusable unit. A role called nginx contains everything needed to install and configure nginx; any playbook can include it with a single line.

13.2 Role Directory Layout

A role is a directory with a fixed structure. Ansible looks in well-known subdirectories for each kind of artefact, so you never need to specify paths in your tasks.

TEXT

```
roles/
  nginx/                    # role name
    tasks/
      main.yml              # main task list (entry point)
    handlers/
      main.yml              # handlers for this role
    defaults/
      main.yml              # default variables (lowest precedence)
    vars/
      main.yml              # role variables (higher than defaults)
    files/
      nginx.conf            # static files for the copy module
    templates/
      nginx.conf.j2         # Jinja2 templates for the template module
    meta/
      main.yml              # role metadata and dependencies
    README.md                # human-readable role documentation
```

```
    tests/
      inventory               # test inventory
      test.yml                # test playbook
```

Subdirectory	Purpose	Auto-loaded?
tasks/	Main task list. main.yml is the entry point.	Yes, main.yml runs by default
handlers/	Handlers for the role. main.yml defines them.	Yes, main.yml loaded
defaults/	Default variables (lowest precedence). Sensible defaults.	Yes, main.yml loaded
vars/	Role variables (higher precedence than defaults).	Yes, main.yml loaded
files/	Static files referenced by copy module.	On demand: copy: src=foo.conf looks here
templates/	Jinja2 templates referenced by template module.	On demand: template: src=foo.j2 looks here
meta/	Role metadata, author info, dependencies.	Yes, main.yml loaded
modules/	Custom Python modules for this role.	Auto-added to module path
tests/	Test inventory and test playbook (convention).	No (used by molecule)

Table 13.1 — Role directory structure

13.3 Creating a Role with ansible-galaxy init

The ansible-galaxy init command scaffolds a new role with the standard directory structure and placeholder files.

BASH

```
cd ~/ansible-lab
ansible-galaxy init roles/nginx

# Verify the structure
ls -la roles/nginx/
```

EXPECTED OUTPUT

```
total 56
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 defaults
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 files
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 handlers
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 meta
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 tasks
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 templates
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 tests
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 24 10:30 vars
-rw-r--r-- 1 sysadmin sysadmin 1325 Oct 24 10:30 README.md
```

13.4 Building a Reusable nginx Role

Let us build a complete, reusable nginx role. Edit each file as shown below.

defaults/main.yml — Sensible defaults

YAML

```
---
# defaults/main.yml
nginx_port: 80
nginx_user: nginx
nginx_worker_processes: auto
nginx_worker_connections: 1024
nginx_doc_root: /usr/share/nginx/html
nginx_index_file: index.html
nginx_install_state: present    # present | latest | absent
```

tasks/main.yml — The main task list

YAML

```
---
# tasks/main.yml
- name: Install nginx (RedHat family)
  ansible.builtin.yum:
    name: nginx
    state: "{{ nginx_install_state }}"
  when: ansible_os_family == 'RedHat'

- name: Install nginx (Debian family)
  ansible.builtin.apt:
    name: nginx
    state: "{{ nginx_install_state }}"
    update_cache: yes
  when: ansible_os_family == 'Debian'

- name: Ensure nginx doc root exists
  ansible.builtin.file:
    path: "{{ nginx_doc_root }}"
    state: directory
    owner: "{{ nginx_user }}"
    group: "{{ nginx_user }}"
    mode: '0755'

- name: Deploy index.html from template
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ nginx_doc_root }}/index.html"
    owner: "{{ nginx_user }}"
    group: "{{ nginx_user }}"
    mode: '0644'
  notify: restart nginx

- name: Deploy nginx.conf from template
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
    backup: yes
  notify: restart nginx
```

```
- name: Open firewall port
  ansible.posix.firewalld:
    service: http
    permanent: yes
    state: enabled
    immediate: yes
  when: ansible_os_family == 'RedHat'

- name: Ensure nginx is started and enabled
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes
```

handlers/main.yml

YAML

```
---
# handlers/main.yml
- name: restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

templates/index.html.j2

HTML

```
<!DOCTYPE html>
<html>
<head><title>Welcome to {{ ansible_hostname }}</title></head>
<body>
<h1>Hello from {{ ansible_hostname }}</h1>
<p>OS: {{ ansible_distribution }} {{ ansible_distribution_version }}</p>
<p>Managed by Ansible role: nginx</p>
</body>
</html>
```

templates/nginx.conf.j2

NGINX

```
user {{ nginx_user }};
worker_processes {{ nginx_worker_processes }};

events {
    worker_connections {{ nginx_worker_connections }};
}

http {
    server {
        listen {{ nginx_port }};
        server_name {{ ansible_hostname }};
        root {{ nginx_doc_root }};
        index {{ nginx_index_file }};
    }
}
```

13.5 Using a Role in a Playbook

There are three ways to use a role in a playbook: roles: (static, loaded at parse time), import_role (also static), and include_role (dynamic, loaded at run time). For most uses, roles: is sufficient.

YAML

```yaml
---
# Method 1: roles keyword (most common)
- name: Apply nginx role to webservers
  hosts: webservers
  become: true
  roles:
    - nginx

# Method 2: with variables overridden
- name: Apply nginx role with custom port
  hosts: webservers
  become: true
  vars:
    nginx_port: 8080
  roles:
    - nginx

# Method 3: include_role (dynamic, can be conditional)
- name: Conditional role
  hosts: webservers
  become: true
  tasks:
    - name: Include nginx role only on webserver1
      ansible.builtin.include_role:
        name: nginx
      when: inventory_hostname == 'webserver1'
```

13.6 Role Dependencies

A role can declare dependencies on other roles in meta/main.yml. Dependencies are executed before the role itself, in reverse order (deepest dependency first). This lets you compose small roles into larger configurations.

YAML

```yaml
---
# roles/webapp/meta/main.yml
dependencies:
  - role: nginx
    vars:
      nginx_port: 8080
  - role: postgresql
    vars:
      pg_version: '15'

galaxy_info:
  author: Senior Linux Sysadmin
  description: Web application stack
  license: MIT
  min_ansible_version: '2.16'
  platforms:
    - name: EL
```

```
      versions: ['9']
```

13.7 Ansible Galaxy — The Role Marketplace

Ansible Galaxy (galaxy.ansible.com) is the public marketplace for Ansible content. It hosts thousands of community-contributed roles and collections that you can install and reuse. Always evaluate the quality and maintenance status of a Galaxy role before depending on it in production — check the last commit date, the number of stars, the issue tracker, and the test coverage.

Searching and installing a role

BASH

```
# Search for nginx roles on Galaxy
ansible-galaxy search nginx

# Install a role by name
ansible-galaxy install geerlingguy.nginx

# Install to a specific path (default: /etc/ansible/roles or ~/.ansible/roles)
ansible-galaxy install geerlingguy.nginx -p ./roles/

# List installed roles
ansible-galaxy list

# Remove a role
ansible-galaxy remove geerlingguy.nginx
```

requirements.yml — Managing Role Dependencies

For reproducible installs, list all external roles in a requirements.yml file. Then a single command installs them all. This is the standard way to manage role dependencies in version control.

YAML

```
# ~/ansible-lab/requirements.yml
---
roles:
  - src: geerlingguy.nginx
    version: '3.1.0'
  - src: geerlingguy.mysql
    version: '4.3.0'
  - src: https://github.com/example/custom-role.git
    scm: git
    name: custom-role
    version: main
```

BASH

```
# Install all roles listed in requirements.yml
ansible-galaxy install -r requirements.yml -p ./roles/
```

13.8 Roles vs Collections

Since Ansible 2.10, the recommended packaging unit is the **collection**, which can contain roles, modules, and plugins together. Roles are still fully supported and are simpler for small units of work. Collections are better for distributing a coherent package (like community.docker which includes Docker-related modules, plugins, and roles).

BASH

```
# Install a collection
ansible-galaxy collection install community.docker

# List installed collections
ansible-galaxy collection list
```

MALAYALAM NOTE · Malayalam Teaching Note

Roles vachu nammal reusable units create cheyyum — tasks, handlers, vars, files, templates ellam oru directory-l. ansible-galaxy init vachu scaffold cheyyam. Playbook-l roles: keyword vachu use cheyyam. defaults/ lowest precedence, vars/ higher precedence. meta/main.yml-l dependencies define cheyyum. Ansible Galaxy (galaxy.ansible.com) public marketplace aanu — community roles install cheyyam. requirements.yml vachu version-controlled dependencies manage cheyyam.

CHAPTER SUMMARY

Roles package related tasks, handlers, variables, files, and templates into a reusable directory structure. Create them with ansible-galaxy init. Use them in playbooks via the roles keyword, import_role (static), or include_role (dynamic). Role dependencies are declared in meta/main.yml. Ansible Galaxy is the public marketplace for community roles and collections; manage external dependencies with requirements.yml and ansible-galaxy install -r.

REVIEW QUESTIONS

1. List the eight standard subdirectories of an Ansible role and the purpose of each.
2. Which subdirectory holds variables with the lowest precedence in a role?
3. Write the command to scaffold a new role called apache.
4. Describe the three ways to use a role in a playbook.
5. How do you declare that a role depends on another role?
6. Write a requirements.yml that installs geerlingguy.nginx version 3.1.0.
7. What is the difference between a role and a collection?

CHAPTER 14

Ansible Vault — Securing Secrets

LEARNING OBJECTIVES

- *Understand why Ansible Vault is necessary for sensitive data.*
- *Create, edit, view, and decrypt encrypted files with ansible-vault.*
- *Use encrypt_string for inline secret encryption.*
- *Use vault password files for non-interactive runs.*
- *Integrate Vault securely with CI/CD pipelines.*

14.1 The Problem: Plaintext Secrets in Playbooks

Many playbooks need access to sensitive data — database passwords, API tokens, SSH keys, SSL private keys. Storing these in plain text in your playbook files or group_vars is dangerous: anyone with read access to your Git repository can read them, and backups, logs, and terminal scrollback may capture them. **Ansible Vault** solves this by encrypting individual files or string values with AES-256, so the encrypted content is safe to commit to version control.

14.2 Creating an Encrypted File

The ansible-vault create command creates a new encrypted file. It prompts for a vault password (which you must remember — there is no recovery), then opens your editor. You write YAML variables as usual; on save, the file is encrypted in place.

BASH

```
cd ~/ansible-lab
ansible-vault create group_vars/webservers/vault.yml

# You will be prompted:
# New Vault password:
# Confirm New Vault password:
# (then your editor opens)
```

Inside the editor, write normal YAML:

YAML

```
---
db_password: SuperSecret123!
api_token: ghp_abc123def456
ssl_private_key: |
  -----BEGIN PRIVATE KEY-----
  MIIE... (your full key here)
  -----END PRIVATE KEY-----
```

When you save and exit, the file is encrypted. The contents look like:

EXPECTED OUTPUT

```
$ANSIBLE_VAULT;1.1;AES256
66386437353136623936623633343338623836393366386234373038663434303062303233396232
34363832366634333537376565306435376361306363383766623965306463353062313433326133
36306339333336383436393563353864636333306630393566623631663232353361663334303565
... (many more lines of base64-encoded ciphertext)
```

14.3 Viewing an Encrypted File

BASH

```
ansible-vault view group_vars/webservers/vault.yml

# Prompts for the vault password, then prints the decrypted contents to stdout
```

14.4 Editing an Encrypted File

BASH

```
ansible-vault edit group_vars/webservers/vault.yml

# Prompts for the vault password, then opens the decrypted file in your editor.
# On save, the file is re-encrypted.
```

14.5 Encrypting an Existing File

BASH

```
# Encrypt an existing plaintext file in place
ansible-vault encrypt group_vars/webservers/secrets.yml
```

14.6 Decrypting an Encrypted File

BASH

```
# Decrypt an encrypted file (in place)
ansible-vault decrypt group_vars/webservers/vault.yml
```

SECURITY · Be careful with decrypt in place

ansible-vault decrypt removes the encryption from the file. If you do this in a directory tracked by Git, you may accidentally commit the plaintext version. Prefer ansible-vault view for one-off inspection, or copy the file outside the repo before decrypting.

14.7 Changing the Vault Password (rekey)

BASH

```
ansible-vault rekey group_vars/webservers/vault.yml

# Prompts for the OLD password, then the NEW password (twice)
```

14.8 encrypt_string — Inline Secret Encryption

Sometimes you want to encrypt just one secret value within a larger unencrypted file. ansible-vault encrypt_string generates an encrypted inline value you can paste directly into any YAML file.

BASH

```
# Encrypt a single string
ansible-vault encrypt_string 'SuperSecret123!' --name 'db_password'
```

EXPECTED OUTPUT

```
db_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          66386437353136623936623633343338623836393366386234373038663434303062303233396232
          34363832366634333537376565306435376361306363383766623965306463353062313433326133
          ...
Encryption successful
```

Paste this output directly into your playbook or vars file. Ansible decrypts it at run time when the vault password is provided.

14.9 Vault Password Files — Non-Interactive Decryption

Typing the vault password every time you run a playbook is tedious. For non-interactive use (especially CI/CD), store the vault password in a file and pass it with --vault-password-file (or --vault-id).

BASH

```
# Create a vault password file (NEVER commit this!)
echo 'MySuperSecretVaultPass123' > ~/.ansible_vault_pass
chmod 600 ~/.ansible_vault_pass

# Add to .gitignore to be safe
echo '.ansible_vault_pass' >> ~/ansible-lab/.gitignore

# Run playbook non-interactively
ansible-playbook deploy.yml --vault-password-file ~/.ansible_vault_pass

# Or set via environment variable
export ANSIBLE_VAULT_PASSWORD_FILE=~/.ansible_vault_pass
ansible-playbook deploy.yml
```

SECURITY · NEVER commit the vault password file

If you commit the vault password file alongside the encrypted secrets, the encryption provides zero protection — anyone with the repo can decrypt. Always add the password file to .gitignore, and provide the password to CI/CD via a secure secrets manager (GitHub Actions secrets, GitLab CI variables, HashiCorp Vault, AWS Secrets Manager, etc.).

14.10 Multiple Vault Passwords

For environments with different secret tiers (dev, staging, production), you can use multiple vault passwords with --vault-id. Each encrypted file is tagged with a label, and Ansible uses the matching password to decrypt.

BASH

```
# Encrypt a file with the 'production' vault ID
ansible-vault encrypt group_vars/production/vault.yml --vault-id production@~/.vault_prod

# Encrypt a file with the 'dev' vault ID
ansible-vault encrypt group_vars/dev/vault.yml --vault-id dev@~/.vault_dev

# Run playbook with both vault IDs
ansible-playbook site.yml --vault-id production@~/.vault_prod --vault-id dev@~/.vault_dev
```

14.11 CI/CD Integration Best Practices

- **Store the vault password as a CI/CD secret** (GitHub Actions secrets, GitLab CI variables, Jenkins credentials). Never hard-code it in the pipeline definition.
- **Write the secret to a file at job start** so ansible-playbook can read it via --vault-password-file.
- **Use environment variables** like ANSIBLE_VAULT_PASSWORD_FILE to avoid exposing the password in process listings.
- **Clean up the password file at job end** to prevent leakage between jobs.
- **Rotate vault passwords regularly** — quarterly at minimum, immediately after any personnel change.
- **Audit vault file access** in your CI logs — any job that uses the vault password should be clearly logged.

MALAYALAM NOTE · Malayalam Teaching Note

Ansible Vault vachu nammal sensitive data (passwords, API keys, SSL keys) encrypt cheyyum. AES-256 encryption use cheyyum. ansible-vault create, edit, view, decrypt commands. encrypt_string vachu inline encryption cheyyam. Vault password file vachu non-interactive run cheyyam — aanaal password file Git-ilekk ittaruthu! CI/CD-l vault password secrets manager-nnokku edukkam (GitHub Actions secrets, GitLab CI variables, HashiCorp Vault). Multiple vault IDs vachu dev/prod separate passwords use cheyyam.

CHAPTER SUMMARY

Ansible Vault encrypts sensitive files and inline string values with AES-256. Use ansible-vault create/edit/view/decrypt/rekey/encrypt_string. For non-interactive runs, store the vault password in a file (NEVER commit it) and pass via --vault-password-file. Multiple vault passwords (--vault-id) allow tiered secrets (dev, staging, production). In CI/CD, store the vault password in the platform's secrets manager, write it to a file at job start, and clean up at job end.

REVIEW QUESTIONS

1. Why is storing plaintext database passwords in playbooks a bad idea?
2. Write the command to create a new encrypted file called secrets.yml.
3. How do you view an encrypted file without modifying it?
4. What does ansible-vault encrypt_string do? When would you use it?
5. Where should the vault password file live on disk? Where should it NEVER live?
6. How do you provide the vault password non-interactively for CI/CD?
7. Describe a use case for multiple vault IDs (production@, dev@).

CHAPTER 15

Practical Lab Tasks (8 Labs)

This chapter contains eight hands-on laboratory exercises that progressively build your Ansible skills. Each lab specifies prerequisites, step-by-step procedure, validation commands, expected results, and common troubleshooting tips. Perform the labs in order — each builds on concepts and artifacts from the previous ones. All labs assume the three-node lab environment from Chapter 2 with SSH key authentication already configured.

Lab 1 — Inventory Setup & First Ad-Hoc Ping

Prerequisites

- Three-node lab from Chapter 2 with hostnames and /etc/hosts configured.
- Ansible 2.16 installed on the control node per Chapter 3.
- SSH key authentication configured per Chapter 4.

Procedure

BASH

```
cd ~/ansible-lab

# Step 1: Create the inventory file (INI format)
cat > inventory << 'EOF'
[webservers]
webserver1 ansible_host=192.168.1.21
webserver2 ansible_host=192.168.1.22

[webservers:vars]
ansible_user=ansible
ansible_ssh_private_key_file=/home/sysadmin/.ssh/ansible_ed25519
ansible_python_interpreter=/usr/bin/python3
EOF

# Step 2: Verify the inventory
ansible-inventory --graph

# Step 3: Ping all webservers
ansible webservers -m ansible.builtin.ping
```

Expected Result

Both webservers respond with SUCCESS and "ping": "pong". If you see UNREACHABLE, check SSH connectivity and the path to the private key file. If you see FAILED with a Python-related error, ensure Python 3 is installed on the managed node.

Lab 2 — Install and Configure nginx on Webservers

Prerequisites

Lab 1 complete; inventory file in place.

Procedure

BASH

```
# Step 1: Create the playbook
cat > install_nginx.yml << 'EOF'
---
- name: Install and configure nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.yum:
        name: nginx
        state: present

    - name: Ensure nginx is started and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes

    - name: Open HTTP port in firewalld
      ansible.posix.firewalld:
        service: http
        permanent: yes
        state: enabled
        immediate: yes
EOF

# Step 2: Validate syntax
ansible-playbook install_nginx.yml --syntax-check

# Step 3: Dry run
ansible-playbook install_nginx.yml --check --diff

# Step 4: Real run
ansible-playbook install_nginx.yml

# Step 5: Verify idempotency (run again, should report changed=0)
ansible-playbook install_nginx.yml
```

Validation

BASH

```
curl -I http://webserver1/
curl -I http://webserver2/
```

Both should return HTTP/1.1 200 OK (or 403 if no index file exists). The second playbook run should report changed=0, proving idempotency.

Lab 3 — Create Users with SSH Keys via Playbook

Objective

Create three users on both webservers, each with an SSH public key, using a loop in a playbook.

Procedure

BASH

```
cat > create_users.yml << 'EOF'
---
- name: Create application users
  hosts: webservers
  become: true
  vars:
    users:
      - name: alice
        key: "ssh-ed25519 AAAA... alice@control"
      - name: bob
        key: "ssh-ed25519 BBBB... bob@control"
      - name: carol
        key: "ssh-ed25519 CCCC... carol@control"
  tasks:
    - name: Create users with bash shell
      ansible.builtin.user:
        name: "{{ item.name }}"
        shell: /bin/bash
        groups: wheel
        append: yes
      loop: "{{ users }}"

    - name: Add SSH public key for each user
      ansible.posix.authorized_key:
        user: "{{ item.name }}"
        key: "{{ item.key }}"
      loop: "{{ users }}"
EOF

ansible-playbook create_users.yml
```

Validation

BASH

```
ssh alice@webserver1 'whoami'  # should print 'alice'
ssh bob@webserver2 'id'        # should print bob's uid/gid
```

Lab 4 — Use Variables and Templates to Customise index.html per Host

Objective

Generate a custom index.html on each webserver showing that host's name and IP.

Procedure

BASH

```
# Step 1: Create the template
mkdir -p templates
```

```
cat > templates/index.html.j2 << 'EOF'
<!DOCTYPE html>
<html>
<head><title>{{ ansible_hostname }}</title></head>
<body>
<h1>Welcome to {{ ansible_hostname }}</h1>
<p>IP address: {{ ansible_default_ipv4.address }}</p>
<p>OS: {{ ansible_distribution }} {{ ansible_distribution_version }}</p>
<p>Deployed by: Ansible</p>
</body>
</html>
EOF

# Step 2: Create the playbook
cat > deploy_index.yml << 'EOF'
---
- name: Deploy custom index.html
  hosts: webservers
  become: true
  tasks:
    - name: Render and deploy index.html
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: /usr/share/nginx/html/index.html
        owner: nginx
        group: nginx
        mode: '0644'
EOF

ansible-playbook deploy_index.yml
```

Validation

BASH

```
curl http://webserver1/   # should show 'Welcome to webserver1'
curl http://webserver2/   # should show 'Welcome to webserver2'
```

Lab 5 — Conditional Install: httpd on Alma, apache2 on Ubuntu

Objective

Write a single playbook that installs the correct Apache package based on the OS family.

Procedure

BASH

```
cat > install_apache.yml << 'EOF'
---
- name: Install Apache based on OS family
  hosts: webservers
  become: true
  tasks:
    - name: Install httpd on RedHat family
      ansible.builtin.yum:
        name: httpd
        state: present
```

```
      when: ansible_os_family == 'RedHat'

    - name: Install apache2 on Debian family
      ansible.builtin.apt:
        name: apache2
        state: present
        update_cache: yes
      when: ansible_os_family == 'Debian'

    - name: Ensure Apache is running (RedHat)
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: yes
      when: ansible_os_family == 'RedHat'

    - name: Ensure Apache is running (Debian)
      ansible.builtin.service:
        name: apache2
        state: started
        enabled: yes
      when: ansible_os_family == 'Debian'
EOF

ansible-playbook install_apache.yml
```

Validation

Only the relevant tasks run on each host. Check the output for skipping on the irrelevant tasks. Run ansible webservers -m command -a 'systemctl status httpd' to verify the service is active on AlmaLinux hosts.

Lab 6 — Build a Reusable nginx Role

Objective

Package the nginx installation logic from Lab 2 into a reusable role, then use it in a playbook.

Procedure

BASH

```
cd ~/ansible-lab
ansible-galaxy init roles/nginx

# Edit roles/nginx/defaults/main.yml
cat > roles/nginx/defaults/main.yml << 'EOF'
---
nginx_port: 80
nginx_user: nginx
nginx_doc_root: /usr/share/nginx/html
EOF

# Edit roles/nginx/tasks/main.yml
cat > roles/nginx/tasks/main.yml << 'EOF'
---
- name: Install nginx
  ansible.builtin.yum:
```

```
    name: nginx
    state: present

- name: Ensure doc root exists
  ansible.builtin.file:
    path: "{{ nginx_doc_root }}"
    state: directory
    owner: "{{ nginx_user }}"
    mode: '0755'

- name: Deploy index.html from template
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ nginx_doc_root }}/index.html"
  notify: restart nginx

- name: Open HTTP port
  ansible.posix.firewalld:
    service: http
    permanent: yes
    state: enabled
    immediate: yes

- name: Ensure nginx started and enabled
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes
EOF

# Edit roles/nginx/handlers/main.yml
cat > roles/nginx/handlers/main.yml << 'EOF'
---
- name: restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
EOF

# Edit roles/nginx/templates/index.html.j2
cat > roles/nginx/templates/index.html.j2 << 'EOF'
<h1>nginx role on {{ ansible_hostname }}</h1>
<p>Serving on port {{ nginx_port }}</p>
EOF

# Use the role in a playbook
cat > use_role.yml << 'EOF'
---
- name: Apply nginx role
  hosts: webservers
  become: true
  roles:
    - nginx
EOF

ansible-playbook use_role.yml
```

Validation

BASH

```
curl http://webserver1/   # should show 'nginx role on webserver1'
curl http://webserver2/   # should show 'nginx role on webserver2'
```

Lab 7 — Use Vault to Encrypt a Database Password

Objective

Store a database password encrypted with Ansible Vault and use it in a playbook.

Procedure

BASH

```
# Step 1: Create a vault password file
echo 'MyVaultPass123' > ~/.vault_pass
chmod 600 ~/.vault_pass

# Step 2: Create an encrypted vars file
mkdir -p group_vars/webservers
ansible-vault create --vault-password-file ~/.vault_pass group_vars/webservers/vault.yml
# In the editor, type:
#   db_password: SuperSecretDbPass456

# Step 3: Create a playbook that uses the secret
cat > use_secret.yml << 'EOF'
---
- name: Demonstrate vault usage
  hosts: webservers
  become: true
  tasks:
    - name: Show password length (not the password itself!)
      ansible.builtin.debug:
        msg: "DB password is {{ db_password | length }} characters long"
EOF

# Step 4: Run the playbook
ansible-playbook use_secret.yml --vault-password-file ~/.vault_pass
```

Validation

The playbook should print DB password is 20 characters long (or however long your password is). If you see an error about the vault password, double-check the password file path and content.

Lab 8 — Schedule a Cron Job Across the Fleet with Handlers

Objective

Deploy a backup script to both webservers, schedule it via cron, and use a handler to reload crond if the cron file changes.

Procedure

BASH

```
# Step 1: Create the backup script
mkdir -p files
cat > files/backup.sh << 'EOF'
#!/bin/bash
/usr/bin/tar czf /backup/www-$(date +%F).tar.gz /var/www 2>/dev/null
/usr/bin/find /backup -name 'www-*.tar.gz' -mtime +7 -delete
EOF
chmod +x files/backup.sh

# Step 2: Create the playbook
cat > deploy_cron.yml << 'EOF'
---
- name: Deploy backup script and cron job
  hosts: webservers
  become: true
  tasks:
    - name: Ensure /backup directory exists
      ansible.builtin.file:
        path: /backup
        state: directory
        mode: '0755'

    - name: Copy backup script
      ansible.builtin.copy:
        src: files/backup.sh
        dest: /usr/local/bin/backup.sh
        owner: root
        group: root
        mode: '0755'

    - name: Add cron job to run backup nightly at 2 AM
      ansible.builtin.cron:
        name: "nightly www backup"
        minute: "0"
        hour: "2"
        job: "/usr/local/bin/backup.sh"
      notify: reload crond

  handlers:
    - name: reload crond
      ansible.builtin.systemd:
        name: crond
        state: reloaded
EOF

ansible-playbook deploy_cron.yml
```

Validation

BASH

```
ansible webservers -m ansible.builtin.command -a 'crontab -l' -b
```

Both hosts should show 0 2 * * * /usr/local/bin/backup.sh with the comment #Ansible: nightly www backup. Re-running the playbook should report changed=0, demonstrating idempotency.

MALAYALAM NOTE · Malayalam Teaching Note

Ithu 8 hands-on labs aanu — Lab 1-l inventory setupum ping, Lab 2-l nginx install, Lab 3-l users create with SSH keys, Lab 4-l templates vachu custom index.html, Lab 5-l OS family conditionals, Lab 6-l reusable nginx role, Lab 7-l vault password encryption, Lab 8-l cron job with handlers. Ellam lab environment-il try cheyyuka — theory mathi alla, hands-on venam.

CHAPTER 16

Capstone Project: LAMP Stack Deployment

This capstone project integrates everything you have learned into a single, realistic deployment: a multi-role Ansible playbook that installs a complete LAMP (Linux, Apache, MySQL, PHP) stack on both webserver1 and webserver2, with vault-secured database credentials, custom virtual hosts, and validation tests. Completing this project demonstrates readiness to apply Ansible in a production-like environment.

16.1 Project Goals

- Install Apache, MySQL, and PHP on both AlmaLinux webservers.
- Configure a custom Apache virtual host with a templated configuration.
- Create a MySQL database and user using vault-encrypted credentials.
- Deploy a PHP info page that connects to the database.
- Open firewall ports 80 and 443.
- Validate the deployment with curl, mysql, and a PHP connection test.

16.2 Architecture

The project uses a multi-role structure. Each role handles one component of the LAMP stack. A top-level playbook orchestrates the roles and applies them to the webservers group.

Role	Purpose	Key Tasks
apache	Install and configure Apache HTTP Server	Install httpd, deploy vhost, open firewall ports, start service
mysql	Install and configure MySQL server	Install mysql-server, set root password from vault, create app DB and user
php	Install PHP and PHP-MySQL extension	Install php, php-mysqli, restart httpd, deploy info.php
app	Deploy the application code	Copy app files, set permissions, deploy config with DB credentials

Table 16.1 — LAMP project role breakdown

16.3 Project Directory Layout

TEXT

```
~/ansible-lab/
  ansible.cfg
  inventory
  site.yml                    # Top-level playbook
  group_vars/
    webservers.yml            # Non-secret variables
    webservers/
```

```
    vault.yml               # Vault-encrypted secrets
  roles/
    apache/
      tasks/main.yml
      handlers/main.yml
      templates/vhost.conf.j2
    mysql/
      tasks/main.yml
      handlers/main.yml
    php/
      tasks/main.yml
      handlers/main.yml
      templates/info.php.j2
    app/
      tasks/main.yml
      files/index.php
```

16.4 Variables Plan

YAML

```yaml
# group_vars/webservers.yml (unencrypted)
---
app_domain: example.com
app_doc_root: /var/www/{{ app_domain }}
mysql_port: 3306
mysql_bind_address: 127.0.0.1
app_db_name: webapp
app_db_user: webapp_user

# group_vars/webservers/vault.yml (encrypted with vault)
---
mysql_root_password: ChangeMeRoot123!
app_db_password: ChangeMeApp123!
```

16.5 The Top-Level Playbook (site.yml)

YAML

```yaml
---
# site.yml
- name: Deploy complete LAMP stack
  hosts: webservers
  become: true
  roles:
    - apache
    - mysql
    - php
    - app
```

16.6 The apache Role

YAML

```yaml
# roles/apache/tasks/main.yml
---
```

```
- name: Install httpd
  ansible.builtin.yum:
    name: httpd
    state: present

- name: Create document root
  ansible.builtin.file:
    path: "{{ app_doc_root }}"
    state: directory
    owner: apache
    group: apache
    mode: '0755'

- name: Deploy virtual host configuration
  ansible.builtin.template:
    src: vhost.conf.j2
    dest: /etc/httpd/conf.d/{{ app_domain }}.conf
  notify: restart httpd

- name: Open firewall ports
  ansible.posix.firewalld:
    service: "{{ item }}"
    permanent: yes
    state: enabled
    immediate: yes
  loop:
    - http
    - https

- name: Ensure httpd is started and enabled
  ansible.builtin.service:
    name: httpd
    state: started
    enabled: yes

# roles/apache/handlers/main.yml
---
- name: restart httpd
  ansible.builtin.service:
    name: httpd
    state: restarted

# roles/apache/templates/vhost.conf.j2
<VirtualHost *:80>
    ServerName {{ app_domain }}
    DocumentRoot {{ app_doc_root }}
    <Directory {{ app_doc_root }}>
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog /var/log/httpd/{{ app_domain }}_error.log
    CustomLog /var/log/httpd/{{ app_domain }}_access.log combined
</VirtualHost>
```

16.7 The mysql Role

YAML

```yaml
# roles/mysql/tasks/main.yml
---
- name: Install MySQL server
  ansible.builtin.yum:
    name: mysql-server
    state: present

- name: Ensure mysqld is started and enabled
  ansible.builtin.service:
    name: mysqld
    state: started
    enabled: yes

- name: Set MySQL root password
  community.mysql.mysql_user:
    name: root
    password: "{{ mysql_root_password }}"
    login_unix_socket: /var/lib/mysql/mysql.sock

- name: Create application database
  community.mysql.mysql_db:
    name: "{{ app_db_name }}"
    state: present
    login_user: root
    login_password: "{{ mysql_root_password }}"

- name: Create application DB user
  community.mysql.mysql_user:
    name: "{{ app_db_user }}"
    password: "{{ app_db_password }}"
    priv: "{{ app_db_name }}.*:ALL"
    state: present
    login_user: root
    login_password: "{{ mysql_root_password }}"
```

16.8 The php Role

YAML

```yaml
# roles/php/tasks/main.yml
---
- name: Install PHP and PHP-MySQL extension
  ansible.builtin.yum:
    name:
      - php
      - php-mysqli
      - php-pdo
    state: present
  notify: restart httpd

- name: Deploy info.php
  ansible.builtin.template:
    src: info.php.j2
    dest: "{{ app_doc_root }}/info.php"
    owner: apache
```

```
    group: apache
    mode: '0644'

# roles/php/handlers/main.yml
---
- name: restart httpd
  ansible.builtin.service:
    name: httpd
    state: restarted

# roles/php/templates/info.php.j2
<?php
phpinfo();
echo "<hr>Connecting to MySQL...<br>";
$conn = new mysqli('127.0.0.1', '{{ app_db_user }}', '{{ app_db_password }}', '{{ app_db_name
}}');
if ($conn->connect_error) { die('Connection failed: ' . $conn->connect_error); }
echo 'Connected to MySQL successfully. Server info: ' . $conn->server_info;
$conn->close();
?>
```

16.9 The app Role

YAML

```
# roles/app/tasks/main.yml
---
- name: Deploy index.php
  ansible.builtin.copy:
    src: index.php
    dest: "{{ app_doc_root }}/index.php"
    owner: apache
    group: apache
    mode: '0644'

# roles/app/files/index.php
<!DOCTYPE html>
<html>
<head><title>LAMP Stack Demo</title></head>
<body>
<h1>LAMP Stack Successfully Deployed</h1>
<p>Host: <?php echo gethostname(); ?></p>
<p><a href="/info.php">PHP Info & MySQL Connection Test</a></p>
</body>
</html>
```

16.10 Running the Capstone

BASH

```
cd ~/ansible-lab

# Validate syntax
ansible-playbook site.yml --syntax-check

# Dry run with diff
ansible-playbook site.yml --check --diff --vault-password-file ~/.vault_pass
```

```
# Real run
ansible-playbook site.yml --vault-password-file ~/.vault_pass
```

16.11 Validation Tests

BASH

```
# Test 1: HTTP returns the LAMP demo page
curl http://webserver1/
curl http://webserver2/

# Test 2: PHP info page renders and connects to MySQL
curl http://webserver1/info.php | grep -i 'Connected to MySQL'
curl http://webserver2/info.php | grep -i 'Connected to MySQL'

# Test 3: MySQL connection from command line
ansible webservers -m community.mysql.mysql_info -a \
  'login_user={{ app_db_user }} login_password={{ app_db_password }} login_host=127.0.0.1' \
  -e '@group_vars/webservers.yml' -e '@group_vars/webservers/vault.yml' \
  --vault-password-file ~/.vault_pass
```

16.12 Cleanup

BASH

```
cd ~/ansible-lab
cat > teardown.yml << 'EOF'
---
- name: Tear down LAMP stack
  hosts: webservers
  become: true
  tasks:
    - name: Remove app document root
      ansible.builtin.file:
        path: "{{ app_doc_root }}"
        state: absent

    - name: Remove vhost config
      ansible.builtin.file:
        path: /etc/httpd/conf.d/{{ app_domain }}.conf
        state: absent

    - name: Stop and disable services
      ansible.builtin.service:
        name: "{{ item }}"
        state: stopped
        enabled: no
      loop: [httpd, mysqld]
      ignore_errors: yes

    - name: Remove packages
      ansible.builtin.yum:
        name: [httpd, mysql-server, php, php-mysqli, php-pdo]
        state: absent
EOF
ansible-playbook teardown.yml
```

MALAYALAM NOTE · Malayalam Teaching Note

Ithu capstone project aanu — nammal adhu vare padichathellam oru realistic deployment-il integrate cheyyum. LAMP stack (Linux-Apache-MySQL-PHP) deploy cheyyum 4 roles use cheyyum: apache, mysql, php, app. Vault vachu MySQL root password-um app DB password-um encrypt cheyyum. Templates vachu virtual host config generate cheyyum. Validation tests vachu verify cheyyum. Ithu complete cheyumbol production-ready Ansible skill ready aanu.

CHAPTER 17

Interview Question Bank (40+ Questions)

This chapter contains 45 interview questions organised by category, each with a concise model answer suitable for technical interviews. Review these before any Ansible-related job interview or internal assessment.

A. Basic Concepts (10 Questions)

Q1. What is Ansible and what problem does it solve?

Answer: Ansible is an open-source IT automation tool that simplifies configuration management, application deployment, and orchestration. It solves the problem of manually configuring hundreds of servers by letting administrators describe the desired state in YAML playbooks that Ansible enforces idempotently across all hosts in parallel via SSH.

Q2. Why is Ansible called agentless?

Answer: Ansible does not require any agent software to be installed on managed hosts. It uses standard SSH (Linux) or WinRM (Windows) to push configuration, executing small Python snippets remotely and removing them afterwards. This eliminates the need to maintain agent packages, agent upgrades, and agent master servers.

Q3. What is idempotency and why is it important?

Answer: Idempotency means running an operation multiple times produces the same end state as running it once. Ansible modules are idempotent by design — they check the current state and only make changes if needed. This makes playbooks safe to re-run after failures, network blips, or for verification, without producing duplicate users, files, or packages.

Q4. Explain the difference between push and pull configuration models.

Answer: In the push model, a central control node initiates connections to managed nodes and pushes the desired state. Ansible uses push by default. In the pull model, managed nodes periodically contact a central server to fetch the latest configuration. Puppet and Chef default to pull. Push is simpler and more predictable; pull scales better for very large estates with hosts behind firewalls.

Q5. What language are Ansible playbooks written in?

Answer: YAML (YAML Ain't Markup Language). YAML is human-readable, uses indentation for structure, and supports lists, dictionaries, and scalars. Ansible chose YAML because it is easy for non-programmers to read and write.

Q6. What is the difference between ansible-core and the ansible community package?

Answer: ansible-core is the minimal runtime: the CLI, the engine, and the built-in ansible.builtin modules. The ansible community package bundles ansible-core plus a curated set of community collections (network, cloud, system tools). For minimal installations use ansible-core; for a complete out-of-the-box experience use the community package.

Q7. What is a playbook? What is a play? What is a task?

Answer: A playbook is a YAML file containing one or more plays. A play targets a group of hosts and contains a list of tasks plus play-level directives. A task is a single unit of work that calls one Ansible module with arguments. Hierarchy: playbook > play > task > module call.

Q8. What is the difference between Ansible and Terraform?

Answer: Terraform is for provisioning infrastructure (creating VMs, VPCs, load balancers) — it is declarative for cloud resources. Ansible is for configuring infrastructure that already exists (installing packages, starting services, deploying apps). They are complementary: Terraform creates the servers, Ansible configures them.

Q9. Name three advantages of Ansible over shell scripts.

Answer: (1) Idempotency — Ansible modules check state before changing; shell scripts need explicit guards. (2) Cross-host parallelism — Ansible runs the same task on hundreds of hosts simultaneously. (3) Readable YAML structure with named tasks, handlers, and variables — far more maintainable than long shell scripts.

Q10. What is the Ansible control node and what are managed nodes?

Answer: The control node is the machine where Ansible is installed and from which playbooks are executed. Managed nodes (also called hosts) are the target servers that Ansible configures over SSH. The control node must have Python; managed nodes need Python and SSH but no Ansible installation.

B. Inventory & Ad-Hoc Commands (8 Questions)

Q11. What is an inventory file?

Answer: An inventory file lists the hosts Ansible manages, organised into groups. It can be in INI or YAML format. It also supports per-host and per-group variables. The default location is /etc/ansible/hosts, but a project-level inventory configured in ansible.cfg is recommended.

Q12. What is the difference between [webservers] and [webservers:vars] in an INI inventory?

Answer: [webservers] defines a group and lists its hosts. [webservers:vars] defines variables that apply to every host in the webservers group. There is also [group:children] for groups of groups.

Q13. What are group_vars and host_vars directories?

Answer: group_vars/ contains YAML files named after groups — the variables in group_vars/webservers.yml apply to all hosts in the webservers group. host_vars/ works the same way per-host. Host variables override group variables, which override all-group variables.

Q14. Write an ad-hoc command to install nginx on all webservers using sudo.

Answer: ansible webservers -m ansible.builtin.yum -a 'name=nginx state=present' -b. The -b flag enables sudo. For Debian-family hosts use the apt module instead of yum.

Q15. What is the difference between command and shell modules?

Answer: command runs an executable without invoking a shell — no pipes, redirects, or environment variables. It is safer because it is immune to shell injection. shell runs through /bin/sh, so pipes and redirects work but the input must be sanitised. Prefer command unless you need shell features.

Q16. What does the -v, -vv, -vvv, -vvvv progression do?

Answer: Each increases the verbosity of Ansible output. -v shows task results in detail, -vv adds task paths and module arguments, -vvv adds SSH connection details, -vvvv adds SSH plugin internals for deep debugging.

Q17. How do you target only one host from a group on the command line?

Answer: Use --limit (or -l): ansible webservers --limit webserver1 -m ping. You can also use patterns like webserver[0] or comma-separated lists.

Q18. What does --check do?

Answer: --check performs a dry run — Ansible connects to hosts, gathers facts, evaluates conditionals, and reports what each task would change, but does not actually make changes. Combine with --diff to see line-by-line previews of file changes.

C. Playbooks & YAML (8 Questions)

Q19. What is the structure of a playbook?

Answer: A playbook is a YAML list of plays. Each play has a name:, a hosts: directive, optional play-level directives like become: and vars:, and a tasks: list. Optional handlers: define service restart tasks. Each task has a name and a module call with arguments.

Q20. Why must YAML use spaces instead of tabs?

Answer: The YAML specification forbids tabs for indentation because tab widths vary by editor. Ansible (and most YAML parsers) will reject a file with tab indentation. Always configure your editor to insert spaces when you press Tab.

Q21. What is the difference between vars and vars_files?

Answer: vars: defines variables inline in the playbook. vars_files: loads variables from external YAML files. Use vars for small, playbook-specific values; use vars_files for larger variable sets and for vault-encrypted secrets (so the playbook itself stays unencrypted).

Q22. How do you validate a playbook before running it?

Answer: Use ansible-playbook playbook.yml --syntax-check to parse the YAML and validate module names without executing. For deeper checks, install ansible-lint and run ansible-lint playbook.yml — it catches best-practice violations and deprecated syntax.

Q23. What is the difference between roles: and include_role?

Answer: roles: is a static list — roles are loaded at playbook parse time and run unconditionally. include_role is dynamic — the role is loaded at run time when the task is reached, so it can be conditional, looped, or wrapped in a block/rescue.

Q24. What is the become directive and when would you use it?

Answer: become: true enables privilege escalation (sudo by default) for a play or task. Use it for any task that requires root permissions — installing packages, modifying /etc, starting services. Per-task become: false overrides play-level become: true.

Q25. What is the purpose of name: keys in tasks?

Answer: The name: key provides a human-readable description of what the task does. Ansible prints these names as it runs, making the output self-documenting. Names also appear in --list-tasks output and in the Ansible log. Always write descriptive names; never omit them.

Q26. What is the difference between --check and --syntax-check?

Answer: --syntax-check parses the YAML and validates module names without connecting to any host. --check connects to hosts, gathers facts, evaluates conditionals, and reports what would change — a much deeper test, but slower and requires connectivity.

D. Modules & Variables (7 Questions)

Q27. What is the difference between ansible.builtin.yum and ansible.builtin.dnf?

Answer: yum is for older RHEL/CentOS (<=7). dnf is the successor on RHEL 8+/AlmaLinux/Rocky. The yum module actually uses dnf on dnf-based systems transparently, so for modern AlmaLinux 9 either works, but dnf is preferred for clarity.

Q28. What is the setup module used for?

Answer: The setup module gathers facts about a managed host: OS, IP addresses, memory, disks, CPU, mounted filesystems, and more. These facts are stored as variables prefixed with ansible_ and are available in tasks and templates. Run ansible host -m setup to see all facts.

Q29. What is the register keyword for?

Answer: register: captures the result of a task into a variable you can use later. Registered variables include stdout, stderr, rc (return code), changed, and other fields. Use them in conditionals (when: result.rc == 0), debug output, or as input to subsequent tasks.

Q30. Name three magic variables in Ansible.

Answer: (1) inventory_hostname — the name of the current host as listed in the inventory. (2) groups — a dictionary of all groups and their hosts. (3) hostvars — a dictionary of all hosts and their variables, useful for cross-host lookups.

Q31. What is variable precedence in Ansible?

Answer: Variable precedence is the order in which Ansible resolves variables defined in multiple places. There are 22 levels; the highest is extra vars (-e), and the lowest is role defaults. Common levels in increasing order: role defaults < inventory group_vars < inventory host_vars < playbook vars < vars_files < extra vars.

Q32. What does the | default() filter do?

Answer: The default() Jinja2 filter returns the variable's value if it is defined, otherwise a fallback you specify. Example: {{ port | default(80) }} uses port if defined, else 80. This makes playbooks robust to undefined variables without raising errors.

Q33. What is the difference between copy and template modules?

Answer: copy transfers a file as-is from the control node to the managed host. template renders a Jinja2 template (with variable substitution, conditionals, loops) and then writes the result. Use copy for static files; use template for any file that needs per-host customisation.

E. Roles & Galaxy (5 Questions)

Q34. What is the standard role directory structure?

Answer: A role has subdirectories: tasks/ (main task list), handlers/ (handlers), defaults/ (default vars, lowest precedence), vars/ (role vars, higher than defaults), files/ (static files for copy), templates/ (Jinja2 templates), meta/ (metadata and dependencies), and optionally modules/ and tests/.

Q35. How do you create a new role?

Answer: Use ansible-galaxy init roles/role_name. This scaffolds the standard directory structure with placeholder main.yml files. Then edit tasks/main.yml to add your tasks, defaults/main.yml for default variables, and so on.

Q36. What is Ansible Galaxy?

Answer: Ansible Galaxy (galaxy.ansible.com) is the public marketplace for Ansible content — roles and collections contributed by the community. You can search, install, and reuse Galaxy content with the ansible-galaxy command. Always evaluate quality (last commit, stars, issues) before depending on a Galaxy role in production.

Q37. What is a requirements.yml file?

Answer: requirements.yml lists external role and collection dependencies for a project. Run ansible-galaxy install -r requirements.yml to install them all. This makes dependency installation reproducible and version-controlled.

Q38. How do you declare that one role depends on another?

Answer: List the dependency in meta/main.yml of the dependent role: dependencies: [{ role: nginx }]. Ansible runs dependencies before the role itself, in reverse order (deepest first).

F. Vault & Security (5 Questions)

Q39. What is Ansible Vault and why would you use it?

Answer: Ansible Vault encrypts files or string values with AES-256 so sensitive data (DB passwords, API tokens, SSL keys) can be safely committed to version control. Use it for any secret that must travel with the playbook. The encrypted file is unreadable without the vault password.

Q40. How do you run a playbook non-interactively with vault-encrypted files?

Answer: Store the vault password in a file (with strict permissions, never committed to Git), then use --vault-password-file ~/.vault_pass (or set the ANSIBLE_VAULT_PASSWORD_FILE environment variable). In CI/CD, write the password to a file at job start from a secrets manager, then clean up at job end.

Q41. What is encrypt_string used for?

Answer: ansible-vault encrypt_string encrypts a single string value, producing inline !vault YAML you can paste into any vars file. This lets you mix encrypted and plaintext variables in the same file — useful when only one or two values are sensitive.

Q42. What are multiple vault IDs useful for?

Answer: Multiple vault IDs let you use different passwords for different environments (dev, staging, production). For example, --vault-id production@~/.vault_prod --vault-id dev@~/.vault_dev decrypts production secrets with one password and dev secrets with another. This limits blast radius if one password leaks.

Q43. Name three security best practices for Ansible automation.

Answer: (1) Use SSH key authentication with passphrases loaded into ssh-agent; avoid password-based SSH. (2) Restrict sudo permissions to specific commands in /etc/sudoers.d/ rather than NOPASSWD: ALL in production. (3) Encrypt all secrets with Ansible Vault and never commit the vault password file to Git.

Q44. What is the difference between ansible.builtin.ping and a normal ICMP ping?

Answer: The Ansible ping module does not send ICMP echo requests. It tests that Ansible can SSH into the host and execute Python. A successful response (pong) means SSH, Python, and the ansible user setup all work correctly. Use the regular ping command for ICMP tests.

Q45. How do you debug a failing playbook?

Answer: Steps: (1) Run with -v or -vv for more output. (2) Add debug: tasks to print variables and registered results. (3) Use --limit onehost to test on a single host. (4) Run --syntax-check to validate YAML. (5) Check SSH connectivity with ansible host -m ping -vvv. (6) Read the failing task's error message carefully — it usually names the missing file, permission, or module.

CHAPTER 18

Viva Voce Questions (30)

These thirty short questions are designed for oral examination (viva) practice. Each question has a concise answer suitable for spoken delivery. Practise answering them aloud — in a viva you must explain concepts clearly without referring to notes.

A. Foundational (10)

V1. Define Ansible in one sentence.

Ans: Ansible is an open-source agentless automation tool that uses YAML playbooks and SSH to configure and orchestrate IT infrastructure idempotently.

V2. What protocol does Ansible use to communicate with Linux managed nodes?

Ans: SSH (port 22 by default). For Windows managed nodes it uses WinRM.

V3. Name the three levels of Ansible hierarchy.

Ans: Playbook (the YAML file), Play (targets a host group), Task (calls one module).

V4. What does idempotent mean?

Ans: Running an operation multiple times has the same effect as running it once. The system ends in the same state regardless of how many times you run.

V5. Which file format are playbooks written in?

Ans: YAML (YAML Ain't Markup Language).

V6. Name the three sections of ansible.cfg we configured in Chapter 3.

Ans: [defaults], [privilege_escalation], [ssh_connection].

V7. What is the difference between a control node and a managed node?

Ans: Control node runs Ansible and initiates SSH. Managed node is the target server being configured; it needs Python and SSH but no Ansible installation.

V8. What is the default SSH user configured in our ansible.cfg?

Ans: ansible (we set remote_user = ansible in the [defaults] section).

V9. Name the four levels of ansible.cfg precedence.

Ans: (1) ANSIBLE_CONFIG env var, (2) ansible.cfg in current directory, (3) ~/.ansible.cfg, (4) /etc/ansible/ansible.cfg.

V10. Which command shows the active Ansible configuration?

Ans: ansible-config dump shows every setting with its source file.

B. Inventory & Commands (10)

V11. What is the difference between [group] and [group:vars] in an INI inventory?

Ans: [group] lists hosts; [group:vars] sets variables that apply to every host in the group.

V12. How do you verify an inventory file?

Ans: Use ansible-inventory --graph for a tree view or ansible-inventory --host for a JSON dump of a host's variables.

V13. Write the ad-hoc pattern to ping all webservers.

Ans: ansible webservers -m ansible.builtin.ping.

V14. What does the -b flag do in an ad-hoc command?

Ans: Enables sudo privilege escalation (become).

V15. What is the difference between command and shell modules?

Ans: command runs without a shell (no pipes, redirects); shell runs through /bin/sh. Prefer command for safety; use shell only when you need shell features.

V16. Which flag performs a dry run?

Ans: --check (often combined with --diff for file change previews).

V17. What does --limit do?

Ans: Restricts the playbook run to a subset of hosts from the inventory. Example: --limit webserver1.

V18. What is the raw module used for?

Ans: Running commands on hosts that do not have Python installed yet — typically for bootstrapping fresh servers.

V19. How do you check documentation for a module offline?

Ans: ansible-doc (e.g. ansible-doc ansible.builtin.yum).

V20. Name the four verbosity flags and what each adds.

Ans: -v (task results), -vv (task paths and module args), -vvv (SSH connection details), -vvvv (SSH plugin internals).

C. Playbooks, Variables, Roles (10)

V21. What is the difference between vars and vars_files?

Ans: vars defines variables inline in the playbook; vars_files loads variables from external YAML files (useful for vault-encrypted secrets).

V22. What does register do?

Ans: Captures the result of a task into a variable you can use later (for conditionals, debug output, or as input to subsequent tasks).

V23. Name three magic variables.

Ans: inventory_hostname, groups, hostvars (also: group_names, ansible_play_hosts).

V24. Which fact tells you the OS family?

Ans: ansible_os_family (values: RedHat, Debian, Suse, etc.).

V25. What does the when keyword do?

Ans: Conditionally executes a task based on a Jinja2 expression (e.g. when: ansible_os_family == 'RedHat').

V26. What is a handler?

Ans: A special task that runs only when notified by another task. Typically used to restart a service after its config file changes. Handlers run once at end of play.

V27. What is the difference between roles: and include_role?

Ans: roles: is static (loaded at parse time, runs unconditionally); include_role is dynamic (loaded at run time, can be conditional or looped).

V28. What is the purpose of defaults/main.yml in a role?

Ans: Holds default values for role variables (lowest precedence). Sensible defaults that users can override.

V29. What is Ansible Vault?

Ans: A feature that encrypts files or string values with AES-256 so sensitive data can be safely committed to version control.

V30. What is the purpose of block/rescue/always?

Ans: It is Ansible's try/except/finally. Tasks in block run normally; rescue runs on failure; always runs regardless. Useful for graceful rollbacks.

CHAPTER 19

Troubleshooting Guide

This chapter catalogues the most common Ansible errors with their symptoms, root causes, and fixes. Bookmark this page — you will refer to it often in your first weeks of using Ansible.

19.1 Connection Errors

Error Message	Likely Cause	Fix
UNREACHABLE! SSH connection refused	SSH service not running on managed node, or firewall blocking port 22	Start sshd: sudo systemctl start sshd. Open port 22 in firewall.
UNREACHABLE! Permission denied (publickey,password)	SSH key not installed on managed node, or wrong key file	Re-run ssh-copy-id. Verify ~/.ssh/config IdentityFile path.
UNREACHABLE! Host key verification failed	host_key_checking is True and the host key is not in known_hosts	Run ssh-keyscan to add the key to known_hosts, or set host_key_checking=False in ansible.cfg (lab only).
UNREACHABLE! Connection timed out	Network unreachable, wrong IP, or host is down	Ping the host. Verify IP in inventory. Check VPN/network.
FAILED! => {"msg": "to use the ssh connection type you must set ansible_ssh_private_key_file"}	No SSH key configured for the host	Set ansible_ssh_private_key_file in inventory or ansible.cfg, or use ssh-agent.

Table 19.1 — Common connection errors

19.2 Sudo / Privilege Errors

Error Message	Likely Cause	Fix
FAILED! => {"msg": "Missing sudo password"}	sudo requires a password but become_ask_pass is False	Run with -K to prompt for sudo password, OR configure NOPASSWD in sudoers.
FAILED! => {"msg": "user is not in the sudoers file"}	ansible user not in sudoers	Add ansible ALL=(ALL) NOPASSWD: ALL to /etc/sudoers.d/ansible using visudo.
FAILED! => {"msg": "sudo: sorry, you must have a tty to run sudo"}	sudo configured to require a TTY (Defaults requiretty)	Comment out "Defaults requiretty" in /etc/sudoers, or set pipelining=False in ansible.cfg.

Table 19.2 — Common sudo errors

19.3 YAML Syntax Errors

Error Message	Likely Cause	Fix
ERROR! Syntax Error while loading YAML	Tabs instead of spaces, wrong indent, or missing colon	Use 2 spaces per indent. Run ansible-playbook --syntax-check to locate the line.
ERROR! conflicting action statements	Two module calls in the same task	A task can only call one module. Split into two tasks.
ERROR! '...' is undefined	Variable referenced but not defined	Define the variable, or use {{ var | default(...) }} to provide a fallback.
ERROR! failed to combine dictionaries, key already exists	Duplicate variable definitions at the same precedence level	Rename one of the variables or merge dictionaries explicitly.

Table 19.3 — Common YAML syntax errors

19.4 Python Errors on Managed Nodes

Ansible requires Python on managed nodes. AlmaLinux 9 ships with Python 3, but Ansible may default to looking for /usr/bin/python which does not exist. Set ansible_python_interpreter=/usr/bin/python3 in the inventory or group_vars to fix this.

Error Message	Likely Cause	Fix
"/usr/bin/python: No such file or directory"	Ansible looking for Python 2 which is not installed	Set ansible_python_interpreter=/usr/bin/python3 in inventory.
ModuleNotFoundError: No module named '...'	Required Python library not installed on managed node	Install the library: yum install python3-PyMySQL, etc. Or use a different module.
Shared connection failed. Missing python3-... package	Required Python binding missing on managed node	Install via yum: e.g. python3-dnf for dnf module.

Table 19.4 — Common Python errors on managed nodes

19.5 Module Errors

Error Message	Likely Cause	Fix
MODULE FAILURE	Module crashed on the remote host — could be Python error, missing dependency, or wrong path	Run with -vvv to see the full traceback. Check the module documentation for required dependencies.
FAILED! => {"changed": false, "msg": "Could not find a module for ..."}	Module not installed; collection missing	Install the collection: ansible-galaxy collection install community.mysql

Error Message	Likely Cause	Fix
FAILED! => {"msg": "The module ... was not found in configured module paths"}	Typo in module name, or wrong collection prefix	Use full FQCN: ansible.builtin.yum, ansible.posix.firewalld, community.mysql.mysql_user.

Table 19.5 — Common module errors

19.6 Idempotency Issues

If a playbook reports changed on every run even though the system is already in the desired state, you have an idempotency problem. Common causes:

- Using the shell or command module without changed_when. These modules always report changed because Ansible cannot know what the command did.
- A file content that includes a timestamp or hostname that changes on every run.
- A template that produces slightly different whitespace each time.
- A task that creates a file with state: touch — touch always updates the file mtime, so it always reports changed. Use copy with content: instead.

19.7 Verbose Debugging Workflow

When a playbook fails, follow this escalation path:

1. Run with -v first. Often the error message becomes clear.
2. If still unclear, run with -vv to see the module arguments.
3. For SSH issues, escalate to -vvv to see the SSH connection log.
4. For deep SSH plugin issues, use -vvvv.
5. Add a debug: var=result task immediately after a failing task to inspect the registered variable.
6. Use --limit onehost to debug on a single host instead of all hosts.
7. If you suspect a fact is wrong, run ansible host -m setup -a 'filter=*keyword*'.

19.8 Common Fact-Gathering Issues

- ansible_distribution is empty: the setup module could not detect the OS. Ensure /etc/os-release exists on the managed node.
- Fact gathering takes too long: set gather_facts: no if you do not need facts, or use fact_caching in ansible.cfg to cache facts across runs.
- ansible_default_ipv4 is wrong on multi-NIC hosts: edit /etc/ansible/facts.d or override the variable in inventory.

MALAYALAM NOTE · Malayalam Teaching Note

Common errors: SSH connection refused (sshd off or firewall), permission denied (key issue), host key verification (use ssh-keyscan), sudo password (set NOPASSWD), YAML syntax (use --syntax-check), /usr/bin/python missing (set ansible_python_interpreter=/usr/bin/python3). Debugging escalation: -v > -vv > -vvv > -vvvv. Idempotency issues: shell module use changed_when, touch module avoid cheyyuka.

CHAPTER SUMMARY

Most Ansible errors fall into five categories: SSH connectivity, sudo permissions, YAML syntax, Python environment on managed nodes, and module dependencies. Use the verbosity flags (-v to -vvvv) to escalate debugging depth. Always run --syntax-check first, then --check, then the real run. For idempotency issues, audit any use of shell/command without changed_when, and avoid state: touch for files you want to be idempotent.

APPENDIX A

Glossary of Ansible Terminology

This glossary defines the most important Ansible terms in alphabetical order. Each definition is followed by a brief Malayalam gloss for quick comprehension.

Term	Definition	Malayalam Gloss
Ad-hoc command	A one-shot Ansible invocation from the command line, used for quick tasks without writing a full playbook. Pattern: ansible -m -a .	*Oru kaariyam quick aayi cheyyan — playbook venam illa.*
Agentless	A characteristic of Ansible: no special software needs to be installed on managed nodes. Ansible uses existing SSH/WinRM.	*Managed node-il onnum install cheyyanda avashyam illa.*
Ansible Galaxy	The public marketplace for Ansible roles and collections at galaxy.ansible.com.	*Community roles-um collections-um share cheyyunna website.*
Become	The family of directives (become, become_user, become_method) that control privilege escalation (sudo by default).	*Sudo vechu root aakunna mechanism.*
Block	A grouping of tasks that supports rescue and always clauses for try/except/finally style error handling.	*Tasks group aakkum — rescue/always vachu error handle cheyyam.*
Callback	A plugin that modifies Ansible output formatting (e.g. yaml callback for prettier output).	*Output format maathram plugin.*
Check mode	A dry-run mode (--check) where Ansible reports what it would change without making changes.	*Dry run — actual change aakilla, preview matram.*
Collection	A distribution format for Ansible content that can include modules, roles, and plugins together. Example: community.mysql.	*Modules + roles + plugins oru package — FQCN use cheyyum.*
Control node	The machine where Ansible is installed and from which playbooks are executed.	*Ansible install cheythirikkunna machine — adhu ninnu run cheyyum.*
Cron module	A module for managing cron jobs on managed hosts idempotently.	*Cron jobs create/manage cheyyunna module.*
Defaults (role)	The lowest-precedence variable location in a role, found in roles//defaults/main.yml.	*Role-le default variables — lowest precedence.*
Diff mode	A flag (--diff) that shows line-by-line diffs of file changes Ansible would make.	*File changes diff aayi kaanikkum.*
FQCN	Fully Qualified Collection Name. The full path to a module, e.g. ansible.builtin.yum.	*Full module name — ansible.builtin.yum pole.*

Term	Definition	Malayalam Gloss
Fact	A variable automatically discovered about a managed host by the setup module. Examples: ansible_distribution, ansible_memtotal_mb.	*System info automatic aayi kittunna variables.*
Forks	The number of hosts Ansible manages in parallel. Default 5; configurable in ansible.cfg.	*Parallel hosts count — default 5.*
Handler	A special task that runs only when notified by another task. Typically used to restart services after config changes.	*Notify cheyumbol matram run aakkunna task — service restart-ku.*
Host	A single managed server listed in the inventory.	*Inventory-l oru server entry.*
Host vars	Variables in the host_vars/ directory that apply to a single host.	*Per-host variables — host_vars/ folder.*
Idempotency	The property that running an operation multiple times produces the same end state as running it once.	*Multiple times run cheyynjalum same result — safe re-run.*
Inventory	A file (INI or YAML) listing the hosts Ansible manages, organised into groups.	*Managed servers-le list — groups aayi organise cheyyum.*
Jinja2	The Python templating engine Ansible uses for templates and expression evaluation.	*Templating language — {{ var }}, {% if %}, {% for %}.*
Loop	A keyword that repeats a task for each item in a list. Each item is accessible as {{ item }}.	*List-il list items-ekku task repeat cheyyum.*
Magic variable	A special variable automatically provided by Ansible (inventory_hostname, groups, hostvars).	*Auto-provided variables — user define cheyyanda.*
Managed node	A server that Ansible configures. Needs Python and SSH but no Ansible installation.	*Ansible target server — agent onnum venam.*
Module	A small piece of Python code that performs a single task (install package, create user) idempotently.	*Single task cheyyunna Python code — idempotent.*
Notify	A keyword on a task that triggers a handler if the task reports changed.	*Task change aakumbol handler-e trigger cheyyum.*
Play	A section of a playbook that targets a host group and contains a list of tasks.	*Playbook-le oru section — host group + tasks.*
Playbook	A YAML file containing one or more plays that describes the desired state of managed hosts.	*YAML file — desired state describe cheyyum.*
Plugin	Code that extends Ansible functionality beyond modules (callbacks, filters, lookups, etc.).	*Ansible extend cheyyunna code — modules-illatha type.*
Registered variable	A variable populated by the register: keyword with the result of a task.	*Task output capture cheyyunna variable.*

Term	Definition	Malayalam Gloss
Role	A reusable package of tasks, handlers, variables, files, and templates with a standard directory structure.	*Reusable package — tasks + handlers + vars + files + templates.*
Role dependencies	Other roles that must run before this role. Declared in meta/main.yml.	*Role-e munne run aavanam — meta/main.yml-l define.*
Setup module	The module that gathers facts about a managed host at the start of a play.	*System facts gather cheyyunna module.*
SSH agent	A background process that holds decrypted SSH private keys in memory, avoiding repeated passphrase entry.	*SSH keys memory-l vechu passphrase avoid cheyyum.*
Tags	Labels on tasks that allow running subsets of a playbook via --tags or --skip-tags.	*Tasks-ile labels — subsets run cheyyam.*
Task	A single unit of work in a play that calls one module with arguments.	*Oru module call — play-le oru unit.*
Template	A Jinja2 file (.j2) rendered with variables and deployed to managed hosts via the template module.	*Jinja2 file — variable substitution vechu render cheyyum.*
vars_files	A play-level directive that loads variables from external YAML files.	*External YAML file-nokku variables load cheyyum.*
vars_prompt	A play-level directive that prompts the user for variable values at run time.	*Run time-il user-nokku variable tharunnu.*
Vault	The Ansible feature that encrypts files and string values with AES-256 for safe storage of secrets.	*Secrets encrypt cheyyum — AES-256.*
When	A keyword that conditionally executes a task based on a Jinja2 expression.	*Condition-based task execution.*
with_items	Older syntax for loops. Prefer the loop keyword in new code.	*Loop-nte older syntax — loop prefer cheyyam.*

APPENDIX B

Quick Reference Cheat Sheet

This appendix is a printable one-stop reference for the most-used Ansible commands, modules, and syntax. Print it out and keep it next to your workstation.

B.1 Essential CLI Commands

Command	Purpose
ansible --version	Show Ansible version and active config file
ansible-config dump	Show all effective settings with sources
ansible -m ping	Test SSH+Python connectivity
ansible-inventory --graph	Show inventory as a tree
ansible-inventory --host	Show all variables for one host as JSON
ansible-doc	Show module documentation offline
ansible-doc -l | grep keyword	Search available modules
ansible-playbook playbook.yml	Run a playbook
ansible-playbook playbook.yml --check	Dry run (no changes)
ansible-playbook playbook.yml --diff	Show file change diffs
ansible-playbook playbook.yml --syntax-check	Validate YAML without running
ansible-playbook playbook.yml --tags X	Run only tagged tasks
ansible-playbook playbook.yml --limit host	Run on a single host
ansible-playbook playbook.yml --list-tasks	List tasks without running
ansible-galaxy init roles/name	Scaffold a new role
ansible-galaxy install role_name	Install a Galaxy role
ansible-galaxy install -r requirements.yml	Install all dependencies
ansible-vault create file.yml	Create encrypted file
ansible-vault edit file.yml	Edit encrypted file
ansible-vault view file.yml	View encrypted file
ansible-vault encrypt file.yml	Encrypt existing file

Command	Purpose
ansible-vault decrypt file.yml	Decrypt file in place
ansible-vault encrypt_string "val" --name key	Inline encrypt a string

B.2 Top 20 Modules (FQCN)

Module	One-line Use
ansible.builtin.ping	Test SSH+Python connectivity
ansible.builtin.command	Run a command (no shell features)
ansible.builtin.shell	Run a command with shell (pipes, redirects)
ansible.builtin.raw	Run a command without Python (bootstrap)
ansible.builtin.yum	Install RPM packages (AlmaLinux, RHEL, Rocky)
ansible.builtin.apt	Install .deb packages (Ubuntu, Debian)
ansible.builtin.service	Manage services (start/stop/restart/enable)
ansible.builtin.systemd	Manage systemd services directly
ansible.builtin.user	Create/manage users
ansible.builtin.group	Create/manage groups
ansible.builtin.file	Set file properties (owner, mode, state)
ansible.builtin.copy	Copy a file from control to managed
ansible.builtin.template	Render a Jinja2 template and deploy
ansible.builtin.lineinfile	Manage one line in a file (idempotent)
ansible.builtin.blockinfile	Manage a block of lines in a file
ansible.builtin.fetch	Pull a file from managed to control
ansible.builtin.setup	Gather facts (system information)
ansible.builtin.debug	Print messages or variables (debugging)
ansible.builtin.git	Clone or update a git repository
ansible.builtin.cron	Manage cron jobs
ansible.posix.firewalld	Manage firewalld rules (RHEL family)
ansible.posix.synchronize	rsync wrapper for efficient file sync

B.3 Top 10 ansible-playbook Flags

Flag	Purpose
--check	Dry run; report changes without making them
--diff	Show line-by-line file diffs (combine with --check)
--syntax-check	Validate YAML without executing
--limit HOST	Restrict to a subset of hosts
--tags TAG	Run only tasks with this tag
--skip-tags TAG	Run all tasks except those with this tag
--list-tasks	List tasks without running
--list-hosts	List hosts that would be targeted
-v, -vv, -vvv, -vvvv	Increase verbosity for debugging
-e "key=value"	Set extra vars (highest precedence)

B.4 Top 10 Jinja2 Filters

Filter	Example
default(x)	{{ port | default(80) }}
upper / lower	{{ name | upper }}
replace(a, b)	{{ msg | replace('old', 'new') }}
regex_replace(pat, repl)	{{ ip | regex_replace('^([0-9]+)\..*', '\\1') }}
length	{{ users | length }}
join(sep)	{{ hosts | join(',') }}
to_nice_json	{{ config | to_nice_json }}
from_json	{{ json_string | from_json }}
dict2items	{{ mydict | dict2items }}
password_hash(algo)	{{ 'secret' | password_hash('sha512') }}

B.5 Common Magic Variables & Facts

Variable / Fact	Meaning
inventory_hostname	Name of current host in inventory
groups	Dictionary of all groups and their hosts
group_names	List of groups the current host belongs to
hostvars	Dictionary of all hosts and their variables
ansible_play_hosts	List of hosts in the current play
ansible_distribution	OS name (AlmaLinux, Ubuntu, etc.)
ansible_os_family	OS family (RedHat, Debian, Suse, etc.)
ansible_hostname	Short hostname of managed node
ansible_default_ipv4.address	Default IPv4 address
ansible_memtotal_mb	Total RAM in MB
ansible_processor_vcpus	Number of CPU cores
ansible_kernel	Kernel version string

B.6 Essential ansible.cfg Sections

INI

```
[defaults]
inventory = ./inventory
remote_user = ansible
forks = 5
host_key_checking = False      # lab only; True in production
stdout_callback = yaml
roles_path = ./roles

[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False

[ssh_connection]
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
pipelining = True
```

B.7 Useful Ad-Hoc One-Liners

BASH

```
# Ping all hosts
ansible all -m ping

# Check uptime on webservers
```

```
ansible webservers -m command -a 'uptime' -b

# Install nginx on all webservers
ansible webservers -m yum -a 'name=nginx state=present' -b

# Reboot all webservers and wait
ansible webservers -m reboot -b

# Show IPv4 address of all hosts
ansible all -m setup -a 'filter=ansible_default_ipv4'

# Create user 'appuser' on all hosts
ansible webservers -m user -a 'name=appuser shell=/bin/bash groups=wheel append=yes' -b

# Open firewall port 443 on webservers
ansible webservers -m ansible.posix.firewalld -a 'port=443/tcp permanent=yes state=enabled
immediate=yes' -b
```

B.8 Minimal Playbook Template

YAML

```yaml
---
- name: Example playbook
  hosts: webservers
  become: true
  vars:
    key: value
  tasks:
    - name: Descriptive task name
      ansible.builtin.module_name:
        arg1: value1
        arg2: value2
      notify: handler_name

  handlers:
    - name: handler_name
      ansible.builtin.service:
        name: some_service
        state: restarted
```

MALAYALAM NOTE · Malayalam Teaching Note

Ithu oru quick reference cheat sheet aanu — print chettu vechu work cheyumbol refer cheyyam. Top commands, top 20 modules, ansible-playbook flags, Jinja2 filters, magic variables, ansible.cfg sections, ad-hoc one-liners, minimal playbook template — ellam oru page-l.